

LEONARDO LIMA DIONIZIO

TCC LEONARDO DIONIZIO

(versão pré-defesa, compilada em 7 de novembro de 2023)

Trabalho apresentado como requisito parcial à conclusão do Curso de Bacharelado em Ciência da Computação, Setor de Ciências Exatas, da Universidade Federal do Paraná.

Área de concentração: *Ciência da Computação*.

Orientador: Eduardo J. Spinosa.

CURITIBA PR

2023

RESUMO

O resumo deve conter no máximo 500 palavras, devendo ser justificado na largura da página e escrito em um único parágrafo¹ com um afastamento de 1,27 cm na primeira linha. O espaçamento entre linhas deve ser de 1,5 linhas. O resumo deve ser informativo, ou seja, é a condensação do conteúdo e expõe finalidades, metodologia, resultados e conclusões.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetur tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3.

¹E também não deve ter notas de rodapé; em outras palavras, não siga este exemplo... ;-)

ABSTRACT

The abstract should be the English translation of the “resumo”, no more, no less.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetur tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Keywords: Keyword 1. Keyword 2. Keyword 3.

LISTA DE FIGURAS

5.1	Jogabilidade do Mundo 1	23
5.2	Jogabilidade do Mundo 2	23
5.3	Jogabilidade do Mundo 3	24
5.4	Jogabilidade do Mundo 4	24
5.5	Jogabilidade do Mundo 5	25
5.6	Jogabilidade do Mundo 6	25
5.7	Jogabilidade do Mundo 7	26
5.8	Jogabilidade do Mundo 8	26
5.9	Jogabilidade do Mundo 1	27
5.10	Jogabilidade do Mundo 2	27
5.11	Jogabilidade do Mundo 3	0
5.12	Jogabilidade do Mundo 4	0
5.62	4º Mundo - Níveis Do CMA-ES - Otimizando Canos	0
5.64	4º Mundo - Níveis Do PSO - Otimizando Canos	0
5.66	4º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos	0
5.68	5º Mundo - Níveis Do CMA-ES - Otimizando Canos	0
5.70	5º Mundo - Níveis Do PSO - Otimizando Canos	0
5.72	5º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos	0
5.73	6º Mundo - Níveis Aleatórios	0
5.13	Jogabilidade do Mundo 5	0
5.14	Jogabilidade do Mundo 6	0
5.15	Jogabilidade do Mundo 7	1
5.16	Jogabilidade do Mundo 8	1
5.17	Tabela Maximizando canos	1
5.18	Tabela Inimigos	2
5.19	Primeiro Mundo.	2
5.20	Primeiro Mundo.	3
5.21	Primeiro Mundo.	3
5.22	segundo Mundo	4
5.23	Segundo Mundo.	4
5.24	Segundo Mundo.	5
5.25	3º Mundo	5
5.26	3º Mundo	6
5.27	3º Mundo	6

5.28	4 Mundo.	7
5.29	4o Mundo	7
5.30	4o Mundo	8
5.31	5 Mundo.	8
5.32	5o Mundo	9
5.33	5o Mundo	9
5.34	6 Mundo.	10
5.35	6o Mundo	10
5.36	6o Mundo	11
5.37	7 Mundo.	11
5.38	7o Mundo	12
5.39	7o Mundo	12
5.40	8 Mundo.	13
5.41	8o Mundo	13
5.42	8o Mundo	14
5.43	1º Mundo - Níveis Aleatórios.	15
5.44	1º Mundo - Níveis Do CMA-ES - Otimizando Canos	16
5.45	1º Mundo - Níveis Do CMA-ES - Otimizando Inimigos	17
5.46	1º Mundo - Níveis Do PSO - Otimizando Canos	18
5.47	1º Mundo - Níveis Do PSO - Otimizando Inimigos	19
5.48	1º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos	20
5.49	2º Mundo - Níveis Aleatórios.	21
5.50	2º Mundo - Níveis Do CMA-ES - Otimizando Canos	22
5.51	2º Mundo - Níveis Do CMA-ES - Otimizando Inimigos	23
5.52	2º Mundo - Níveis Do PSO - Otimizando Canos	24
5.53	2º Mundo - Níveis Do PSO - Otimizando Inimigos	25
5.54	2º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos	26
5.55	3º Mundo - Níveis Aleatórios.	27
5.56	3º Mundo - Níveis Do CMA-ES - Otimizando Canos	28
5.57	3º Mundo - Níveis Do CMA-ES - Otimizando Inimigos	29
5.58	3º Mundo - Níveis Do PSO - Otimizando Canos	30
5.59	3º Mundo - Níveis Do PSO - Otimizando Inimigos	31
5.60	3º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos	32
5.75	6º Mundo - Níveis Do CMA-ES - Otimizando Inimigos	33
5.77	7º Mundo - Níveis Aleatórios.	34

LISTA DE TABELAS

LISTA DE ACRÔNIMOS

DINF	Departamento de Informática
PPGINF	Programa de Pós-Graduação em Informática
UFPR	Universidade Federal do Paraná

LISTA DE SÍMBOLOS

α	alfa, primeira letra do alfabeto grego
β	beta, segunda letra do alfabeto grego
γ	gama, terceira letra do alfabeto grego
ω	ômega, última letra do alfabeto grego
π	pi
τ	Tempo de resposta do sistema
θ	Ângulo de incidência do raio luminoso

SUMÁRIO

1	INTRODUÇÃO	11
1.1	OBJETIVO	12
2	FUNDAMENTAÇÃO TEÓRICA.	13
2.1	REDES NEURAIS ADVERSÁRIAS GENERATIVAS.	13
2.2	CMA-ES	13
2.3	EVOLUÇÃO DE VARIÁVEL LATENTE (LVE)	14
2.4	GERAÇÃO DE CONTEÚDO PROCEDURAL USANDO APRENDIZADO DE MÁQUINA (PCGML)	15
2.5	NSGA-II.	15
2.6	PSO	16
2.7	ALGORITMO A ESTRELA	16
2.8	MARIOGAN	16
3	TRABALHOS RELACIONADOS	18
3.1	GERAÇÃO DE CONTEÚDO PROCEDURAL VIA MACHINE LEARNING UTILIZANDO REDES GENERATIVAS ADVERSÁRIAS	18
4	PROPOSTA	20
4.1	CONJUNTO DE TREINO	20
4.2	ARQUITETURA DO MODELO NEURAL	20
4.3	MÉTODOS PARA LVE	20
4.4	FUNÇÕES OBJETIVOS	21
4.5	AVALIAÇÃO DE CONTEÚDO	21
4.5.1	Classificação de níveis	22
4.5.2	Jogabilidade e performance de jogo	22
5	RESULTADOS - EM CONSTRUÇÃO, APENAS IMAGENS	23
5.1	JOGABILIDADE MAXIMIZANDO CANOS	23
5.2	JOGABILIDADE MAXIMIZANDO INIMIGOS	24
5.3	TABELA DE VALORES OTIMIZANDO CANOS	25
5.4	TABELA DE VALORES OTIMIZANDO INIMIGOS	25
5.5	CLASSIFICAÇÃO DE PADRÕES	26
5.6	NÍVEIS	0
5.6.1	Mundo 1.	0
5.6.2	Mundo 2.	0
5.6.3	Mundo 3.	0
5.6.4	Mundo 4.	0
5.6.5	Mundo 5.	0

5.6.6	Mundo 6.	0
5.6.7	Mundo 7.	0
5.6.8	Mundo 8.	2
	REFERÊNCIAS	36

1 INTRODUÇÃO

No início da adesão aos modelos neurais, sejam eles profundos ou perceptrons, tais modelos se popularizaram inicialmente nas tarefas de regressão e classificação, gerando competições populares como a ImageNet, competição em que o intuito é classificar imagens corretamente dentre 1000 classes. Ainda assim, aproximar uma distribuição só tornou-se possível com o surgimento das Redes Neurais Adversariais (GANs), inicialmente propostas em (Goodfellow et al., 2014). O paradigma do aprendizado adversarial permitiu, através do uso de uma rede geradora e uma rede discriminadora, que uma rede neural profunda se aproxime de uma distribuição de forma a, ao invés de tomar uma decisão em relação a sua entrada, gerar conteúdo que se assemelhe a um conjunto conhecido.

Em (Goodfellow et al., 2014), o intuito desta geração de conteúdo era a geração de imagens. Com o refinamento, este modelo tornou-se popular em outras tarefas, como a transferência de estilo proposta em (Zhu et al., 2020) e a tradução de texto para imagem ilustrada em (Reed et al., 2016). Embora as redes adversariais generativas tenham tornado-se bastante conhecida nestas tarefas relacionadas com imagens, suas aplicações vão muito além destas. Uma das indústrias que muito se beneficia da geração de conteúdo é o ramo dos videojogos. Embora a geração de conteúdo em jogos esteja presente desde o jogo de 1984 *Elite*, ela é muito utilizada para a geração de níveis.

Grandes exemplos do uso de geração procedural são os jogos *Minecraft*, onde cada mapa é gerado em cima de um número aleatório, uma seed, tornando cada mundo diferente do outro - de forma tão diversificada que seria impensável um humano fazer isso -, *No Man's Sky* e o recente *Starfield*, onde planetas e biomas são gerados proceduralmente num ecossistema online multijogador, e jogos do gênero rogue, onde cada vez que o jogador morre pode melhorar sua personagem e mais uma vez enfrentar os desafios do início, porém, diferentes a cada jogada, como *Rogue Legacy* (1 e 2), *Enter The Gungeon* e *Hades*.

A área acadêmica também estuda a geração de conteúdo para jogos, muitos destes estudos ligados a geração de níveis através do aprendizado adversarial, conforme levantado em (Liu et al., 2020) na **Figura X**. Um popular modelo para geração de níveis para o jogo *Super Mario Bros.* foi proposto em (Volz et al., 2018). A grande inovação do artigo que o tornou popular no nicho não foi a arquitetura da GAN em si, mas sim o fato de utilizar o conceito de evolução de variável latente (LVE) para manipular as entradas do gerador de forma que o conteúdo produzido pelo modelo seguisse certo padrão através de um algoritmo genético, o CMA-ES, proposto inicialmente em (Hansen et al., 2003). A demonstração do artigo foi treinando a rede neural proposta apenas com uma única fase e avaliando a qualidade da fase com a função objetivo do algoritmo genético e também através de métricas de um agente A estrela jogando o nível gerado, porém não se preocupa em verificar se estas fases seguem ou não de forma suficiente o

conjunto de fases conhecidas. Outra limitação de (Volz et al., 2018) é o uso apenas do CMA-ES, não se preocupando em outras formas de explorar o espaço latente, ou tentar uma abordagem multi-objetivo, algo sugerido no final do artigo como possível estudo futuro.

1.1 OBJETIVO

Este trabalho utiliza a rede geradora proposta em (Volz et al., 2018) e, além da já treinada rede com uma única fase do jogo disponibilizada pelos autores, o modelo defendido do artigo também foi treinado com as fases de cada um dos 8 mundos do jogo *Super Mario Bros.* (agrupamentos de até três níveis, que tem como objetivo um aumento de dificuldade gradual) no intuito de avaliar tanto a qualidade e diversidade das fases geradas em um conjunto maior de níveis quanto se as redes aproximam ou não de forma diferenciável o conjunto de fases apresentados no treinamento do modelo gerador.

Para além disso, avaliamos a efetividade do algoritmo proposto para a evolução de variáveis latentes, o CMA-ES, em comparação com um algoritmo simples de otimização de partículas (PSO) e também com uma abordagem multi-objetivo através do algoritmo NSGA-II proposto em (?).

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo revisita os tópicos fundamentais deste trabalho, incluindo os princípios de redes neurais adversariais, evolução de variável latente e algoritmos genéticos para exploração do espaço latente com o exemplo do CMA-ES; explora a literatura no tópico de geração de conteúdo procedural utilizando aprendizado de máquina; e apresenta o principal artigo que guiou este trabalho, que utiliza uma DCGAN para gerar níveis do jogo Super Mario Bros.(1985).

2.1 REDES NEURAS ADVERSARIAIS GENERATIVAS

O paradigma do Aprendizado Adversarial e As Redes Neurais Adversariais Generativas (GANs) foram introduzidos por Goodfellow et al. em 2014 como solução para o problema de estimação de máxima verossimilhança, dilema no qual o custo computacional e a complexidade são extremamente altos, podendo chegar a ser computacionalmente inviável a depender do contexto. Enquanto a gama mais popular de modelos profundos na época de publicação do artigo consistiam de apenas uma rede neural de alta profundidade usada em classificação e, em certos casos, regressão, o aprendizado adversarial proposto pelo artigo usufrui de duas MLPs.

Um desses modelos é apelidado de gerador, abreviado como G , enquanto o outro modelo é chamado de discriminador e abreviado como D . O modelo gerador representa uma função diferenciável responsável por aproximar a distribuição de seus dados de treinamento. Para gerar novos dados, a rede G projeta entradas ruidosas no espaço dos dados conhecidos, na intenção de aproximar o conteúdo gerado da distribuição dos conjuntos conhecidos.

Por sua vez, a MLP D tem como objetivo traçar um plano que separe as distribuições do conteúdo original das distribuições das projeções de G . A saída da rede discriminadora, no artigo, é descrita como um único escalar que representa a probabilidade de uma entrada não ser gerada. Essencialmente, G e D tornam-se antagônicas a partir do momento que suas respectivas funções de perda colocam como objetivo penalizar a outra. Para que isso seja possível, G é treinada para minimizar $\log(1 - D(G(z)))$ enquanto D é treinada para maximizar a probabilidade de acertar se uma entrada é sintética ou real. Este treinamento antagônico leva essas duas redes a disputarem um jogo MinMax de dois jogadores. Alternando a otimização de D e G , a rede discriminadora fica mais perto da solução ótima, enquanto a MLP geradora muda aos poucos para sua distribuição aproximar cada vez mais da original e, por consequência, tornar a solução que o discriminador busca mais complexa de se aproximar.

2.2 CMA-ES

CMA-ES, a *Covariance Matrix Adaptation Evolution Strategy*, foi instanciada primeiro em (Hansen et al., 2003). É uma estratégia evolucionária que se comporta bem com vetores de

números reais, que neste casos são as entradas do gerador, e sua principal característica é o uso de uma matriz de covariância estimada iterativamente por diferenças finitas.

Como está no seu nome, o algoritmo faz parte de um grupo de algoritmos evolucionários chamados *evolutionary strategies*, que é um grupo de algoritmos caixa preta caracterizados também pelo uso da ordenação por ranking. A CMA-ES tem se mostrado eficiente para otimizar problemas não-convexos e não-lineares em domínios contínuos.

O que este algoritmo faz é calcular uma gaussiana com base na matriz de covariância estimada. Esta gaussiana é traduzida numa elipse que se desloca de forma a contemplar as melhores soluções. Quanto mais distantes as melhores soluções estão do domínio dessa elipse, maior o *step size* dela, ou seja, seu raio. De certa forma, no começo, o *step size* costuma ser mais alto pois as melhores soluções conhecidas estão esparsas e a partir deste ponto de partida há a exploração global. Conforme a elipse fica melhor posicionada no domínio, as melhores soluções ficarão menos esparsas, fazendo com que o *step size* diminua proporcionalmente com a concentração da população.

Segue um pseudo-código do que seria o CMA-ES:

Algoritmo 1 CMA-ES

Inicialize a população: λ Inicialize o step size: σ Defina o tamanho da população: m defina o número de melhores candidatos μ Defina os pesos: $W_1, W_2, \dots, W_m$ **repeat** $x \in \lambda$ Calcula a gaussiana para x Ordena λ pelo fitness $c \leftarrow \text{calcula_covariância}(\lambda)$ $m \leftarrow \sum_{i=1}^{\mu} W_i \cdot X_i$ $\sigma \leftarrow \text{atualiza}(\sigma)$ **until** número de iterações $> \text{max_iter}$

2.3 EVOLUÇÃO DE VARIÁVEL LATENTE (LVE)

LVE - *Latent Variable Evolution* - surgiu primeiro para buscar digitais específicas num certo espaço mapeado por uma GAN em (Bontrager et al., 2017). Como dito anteriormente, a rede neural geradora de uma GAN mapeia entradas ruidosas para um espaço. Estas entradas ruidosas também são chamadas de variáveis latentes, ou seja, variáveis que são difíceis ou impossível de controlar e interpretar antes de um resultado concreto.

A partir disso, a técnica de LVE tem como ideia manipular as entradas ruidosas de um modelo para que sua saída se encaixe dentro de certas especificações, ou seja, adiciona ao conteúdo gerado restrições que até o momento atual não são controláveis a nível de treinamento, pois uma rede geradora tem como única função reproduzir a distribuição de seus dados de teste. Em (Bontrager et al., 2017), a técnica de LVE é utilizada para a quebra de identificação de digitais. A técnica consiste de duas etapas: primeiro, treinar uma rede neural para gerar o conteúdo alvo e, após isso, usar um algoritmo para buscar no espaço latente uma entrada que gere uma saída dentro das condições especificadas.

O algoritmo utilizado por (Bontrager et al., 2017) para busca no espaço latente é o já explicado anteriormente CMA-ES. A justificativa do uso do CMA-ES pelo autores é que a função

objetivo é não contínua e, pela natureza de redes convolucionais, também não é linearmente separável.

2.4 GERAÇÃO DE CONTEÚDO PROCEDURAL USANDO APRENDIZADO DE MÁQUINA (PCGML)

A survey de Summerville e colegas (Summerville et al., 2018) define *Procedural Content Generation Via Machine Learning* como "geração de conteúdo de jogos usando modelos de *machine learning* em conteúdos existentes". Na área, busca-se aumentar o valor de replay, salvar memória, reduzir o custo e o esforço de produção, diversificar estética e reduzir a presença humana na geração de conteúdo de jogos. Na área acadêmica, além disso, pode-se dizer que se explora novas maneiras de experienciar os jogos, além de ser um desafio que estimula a criatividade computacional e o entendimento de *game design* pela construção de modelos.

2.5 NSGA-II

Em Abril de 2002 foi proposto a evolução do nondominated sorting genetic algorithm (algoritmo genético de ordenção não-dominada), o NSGA-II (Deb, 2002), que hoje em dia conta com 37971 citações (Connected Papers, acesso em 05/11/2023). Esse algoritmo foi proposto na intenção de contornar os três principais problemas dos algoritmos evolucionários apontados na época:

- Alta complexidade computacional (especificamente $O(MN^3)$, onde M é o número de objetivos e N o tamanho da população);
- A falta de elitismo;
- A necessidade de especificar parâmetros, como o de compartilhamento do primeiro NSGA(?);

E sua implementação nos permite:

- Preservar a diversidade;
- e enfatizar soluções não-dominadas.

Tendo populações de tamanho N , o NSGA-II recombina tal população numa população de tamanho $2N$ e a ordena baseado no ranking da fronteira de Pareto, priorizando soluções da fronteira classe 1 e completando com as fronteiras subsequentes. Quando a fronteira F_g não cabe mais inteira na próxima população, são selecionados os indivíduos com maior crowd distance, que é a área/volume (dependente do tamanho do espaço objetivo Z do problema) ao seu redor sem nenhuma solução presente (cabe enfatizar que as funções objetivo são normalizadas para que nenhuma domine a outra). Este operador permite escolher as soluções que estão mais distantes das outras, por consequência também sendo um preservador de diversidade por si só.

2.6 PSO

Otimização por enxames de partículas foi proposta por (?). Segundo os autores, é um método de otimização para funções contínuas e não-lineares que simula um modelo social simplificado. O algoritmo instância uma população candidata e a move pelo domínio de acordo com fórmulas matemáticas simples em relação a posição e velocidade da partícula.

A metaheurística associada ao PSO faz pouca ou nenhuma suposição do problema que será otimizado. Outra vantagem é que o algoritmo não utiliza otimização por gradiente como muitos dos algoritmos de computação evolutiva, o que significa que a função objetivo não precisa ser diferenciável.

2.7 ALGORITMO A ESTRELA

O algoritmo A estrela é uma extensão do Algoritmo de Dijkstra que utiliza alguma metaheurística para guiar a busca pelo caminho mais curto ao objetivo. Para que o algoritmo funcione corretamente, a heurística definida para auxiliar a busca deve ser admissível, isto é, o custo estimado para atingir o estado final em relação ao estado atual deve ser menor ou igual ao custo verdadeiro para atingir o estado final do estado atual.

A busca A estrela possui múltiplas aplicações, entre elas a busca por caminhos viáveis em videogames. Até os dias de hoje, o mais popular algoritmo para o teste de jogabilidade de níveis do jogo

2.8 MARIOGAN

Em maio de 2018 publicou-se o artigo que deu origem ao que outros entusiastas do estudo da PCGML chamam de MarioGAN (Volz et al., 2018). O artigo aborda o uso de GANs, mas não foca na arquitetura da rede geradora ou discriminadora, mas sim utiliza a ideia de LVE para geração de níveis procedurais no jogo *Super Mario Bros*. Aqui, seguindo a ideia de (Bontrager et al., 2017), o CMA-ES é utilizado para adicionar alguma restrição ao espaço latente do gerador, sendo testadas quatro funções-objetivo diferentes, duas delas relacionadas a disposição dos blocos da fase e outras duas em relação ao teste das fases pelo agente A* vencedor da competição Mario AI de 2009(Togelius et al., 2010) de autoria de Robin Baumgarten.

Para representar as fases, os autores utilizaram a representação do jogo *Super Mario Bros* disponível no repositório do *github* aberto da Video Game Level Corpus(VGLC) (Summerville et al., 2016), que, entre suas particularidades, simplificam a representação do jogo ao ignorar níveis específicos, onde a física do jogo diverge do padrão seguido pela maioria das fases, e também representa todos os inimigos da mesa maneira, tornando a tarefa possivelmente mais simples. Nesta representação, cada tipo de bloco é mapeado para um caracter específico, transformando a imagem em uma sequência de caracteres que indicam um bloco específico. Uma

vez que o artigo implementa a arquitetura a partir da biblioteca *Pytorch*, esta representação textual precisou ser remapeada para uma representação numérica, conforme mostrado pela (Figura Y).

A GAN convolucional profunda (DCGAN) utilizada no artigo é baseada na Wassertein Gan apresentada em (Arjovsky et al., 2017), em que o discriminador utiliza convoluções com deslocamento (*strided convolutions*), enquanto o gerador utiliza convoluções com deslocamento fracionário (*fractional-strided convolutions*). A arquitetura em (Volz et al., 2018) difere de (?) nos seguintes pontos:

- Tanto no discriminador quanto no gerador, é empregada uma camada de normalização de bloco (*batch normalization* após cada camada convolucional;
- A função de ativação de todas as camadas (inclusive na de saída) é a ReLU (O discriminador, como em (Arjovsky et al., 2017), utiliza a Leaky ReLU como função de ativação em suas camadas).

Tanto o discriminador quanto o gerador são treinados com o otimizador RMSprop, com um *batch size* de tamanho 32 e a taxa de aprendizado de 0,00005 por 5000 iterações. A arquitetura é devidamente ilustrada na (Figura Y).

O gerador utilizado no artigo é treinado apenas com o primeiro nível do jogo, na intenção de mostrar como a LVE e a GAN treinada poderiam, em conjunto, gerar níveis diferentes e jogáveis. Outra escolha interessante do artigo da MarioGAN (Volz et al., 2018) é que, para gerar múltiplas amostras com apenas uma fase, a representação numérica foi separada com base em uma janela deslizante com dimensões de 14 de altura e 28 de largura, com passo de tamanho um.

Uma noção geral do processo executado por (Volz et al., 2018) está devidamente traduzido na (Figura Z). Este método é dividido em duas fases básicas distintas: primeiro, a WGAN é treinada de maneira não-supervisionada para gerar níveis do *Super Mario Bros*; após isso, o CMA-ES é utilizado de forma a buscar no espaço latente da GAN, ou seja, alterar a entrada do modelo para satisfazer propriedades específicas. Embora o artigo se baseie apenas em níveis do jogo *Super Mario Bros*, os autores clamam que o modelo é suficientemente bom para generalizar qualquer jogo com representação compatível.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta uma visão geral de estudos existentes em relação ao uso de redes generativas adversariais para geração de fases e também do uso da exploração de variáveis latentes para guiar o conteúdo gerado. Ele revisa a definição de geração de conteúdo procedural, apresenta sistemas utilizados para geração de conteúdo e discute modelos de geração de conteúdo relevantes para a literatura.

3.1 GERAÇÃO DE CONTEÚDO PROCEDURAL VIA MACHINE LEARNING UTILIZANDO REDES GENERATIVAS ADVERSARIAIS

Antes mesmo do machine learning chegar a área de geração procedural de conteúdo, vários métodos baseados em busca como n-gramas e cadeias de markov já eram empregados como levantado em (Togelius et al., 2011). Neste artigo de certa popularidade, PCG é definida como "criar conteúdo para jogos automaticamente, através de meios algorítmicos". Já em (Hendrikx et al., 2013), os autores definem PCG-G (geração de conteúdo procedural para jogos) como "a aplicação de computação para gerar conteúdo para jogos, distinguindo instâncias interessantes entre as amostras geradas, e selecionar instâncias interessantes em prol dos jogadores". Embora o termo não tenha se popularizado, (Hendrikx et al., 2013) comenta muito bem vários dos desafios dessa área, com ênfase em como julgar o valor técnico e cultural do conteúdo gerado.

Alguns exemplos de PCGML são o *Bardo Composer* (Ferreira et al., 2020) que gera músicas para RPGs de Mesa, geração de personagens em jogos no estilo pixel-art em (Rebouças Serpa e Formico Rodrigues, 2019), ou até mesmo aplicações mais práticas, como a desenvolvedora *CD Projekt RED*, que fez uma parceria com a empresa canadense *Jali Research* para, através de técnicas híbridas de inteligência artificial clássica e aprendizado de máquina, fazer com que a boca dos personagens se adapte a dublagem correspondente do diálogo, e o recente anúncio da desenvolvedora francesa *Ubisoft* do desenvolvimento de uma ferramenta para ajuda de construção de narrativas de jogos, a *Ubisoft Ghostwriter*.

Porém a geração de conteúdo procedural data ao jogo de 1984 *Elite*, onde os desenvolvedores usavam uma sequência definida de sementes aleatórias para gerar composições de planetas na intenção de economizar memória armazenando apenas um gerador ao invés dos planetas inteiros. De acordo com o gráfico presente em (Liu et al., 2020), quando se fala de PCGML, há uma quantia desproporcional de pesquisa em geração de níveis 2D em comparação aos demais tópicos. Para além disso, (Liu et al., 2020) ilustra no mesmo gráfico o quando o paradigma do aprendizado adversarial domina este tópico. Segundo os autores, o modelo adversarial mais popular neste campo seriam as GANs e suas variações.

Em (Giacomello et al., 2018), algumas GANs combinaram imagens e atributos topológicos e posicionais extraídos de conteúdo feito por humanos para gerar níveis do jogo *Doom*.

Neste trabalho em específico, os níveis não foram testados para checar se são jogáveis ou não. Já em (Torrado et al., 2019), além de propor uma nova arquitetura que acrescenta camadas de *embedding* e módulos de atenção, a pequena quantia de dados de treino do jogo *The Legend Of Zelda (1986)* foi expandida por uma técnica de *bootstrapping*. Alguns dos resultados de (Torrado et al., 2019) estão na (Figura N).

(Capps e Schrum, 2021) utiliza múltiplas GANs, um modelo o qual os autores apelidaram como multiGAN, para gerar níveis melhores conectados do jogo *megaman (1987)*. Para evoluir os níveis a atingirem certa propriedade, ao invés do CMA-ES, os autores utilizam o NSGA-II (*Non-Dominated Sorting Genetic Algorithm*), um algoritmo multi-objetivo baseado em conceitos de Pareto. Para além disso, (Capps e Schrum, 2021) utiliza um algoritmo A estrela para predizer os caminhos e ajudar na mutação das variáveis latentes.

Para além de geração de níveis, GANs também são utilizadas para produzir proceduralmente outros conteúdos. (Hong et al., 2019) propões o uso de dois codificadores (*encoder*) distintos para estrutura, formato e cor, juntamente com discriminadores distintos para formato e cor para geração de silhuetas de personagens. (Jin et al., 2017) também utiliza redes generativas adversariais para gerar personagens, desta vez para o jogo *The Sims (2000)*. Em outro exemplo, (Fadaeddini et al., 2018) utiliza GANs para geração de texturas em jogos.

Como já explicado anteriormente, (Volz et al., 2018) define uma DCGAN baseada na WGAN apresentada por (?) treinada em um único nível do jogo *Super Mario Bros* e inova na forma de adicionar restrições a rede geradora após seu treinamento utilizando a técnica de LVE para adicionar restrições a saída do modelo. Neste artigo, o CMA-ES é o algoritmo utilizado para a busca no espaço latente e é aplicado após o treinamento da rede neural adversarial.

4 PROPOSTA

Este capítulo apresenta os métodos propostos para comparação entre os métodos de LVE, bem como os para avaliação do conteúdo gerado por redes treinadas em diferentes conjuntos. Ele apresenta a arquitetura do modelo neural adversarial utilizado, o conjunto de dados em que o modelo foi treinado e as funções objetivo propostas.

4.1 CONJUNTO DE TREINO

O jogo escolhido para o trabalho é o mesmo presente em (Volz et al., 2018), *Super Mario Bros*. Enquanto (Volz et al., 2018) se propõe a avaliar a diversidade de conteúdo gerado a partir do treinamento do modelo com um único nível (o primeiro), neste trabalho é avaliado o quão diversificados são os conteúdos gerados em conjuntos de granularidade diversa.

Como em (Volz et al., 2018), a representação utilizada é a mesma do repositório aberto da *Video Game Level Corpus* ((Summerville et al., 2016)), em que os blocos estão representados na (FIGURA N). Reproduzindo a divisão do artigo de inspiração, a representação textual é dividida por uma janela deslizante de 14 blocos/caracteres de altura e 28 blocos/caracteres de largura. Além do modelo disponibilizado por (Volz et al., 2018)(colocar a footnote), treinado em uma única fase, nós treinamos redes com os oito mundos (agrupamentos de níveis com dificuldade gradual). A (TABELA A) mostra, em sua totalidade, a quantidade de níveis processados em cada conjunto de treino, bem como a quantidade de janelas obtidas.

4.2 ARQUITETURA DO MODELO NEURAL

Neste trabalho, a arquitetura do modelo de (Volz et al., 2018) foi preservada, arquitetura esta devidamente ilustrada na(FIGURA X). A principal motivação para tal preservação é para que a comparação entre conjuntos de treinamento e métodos de busca em espaço latente seja justa. Embora seja possível facilmente trocar o modelo no fluxo de dados (assim como é possível trocar o modelo de LVE), este não é o foco deste trabalho. O código utilizado no modelo é o mesmo disponibilizado por (Volz et al., 2018), com sua implementação com auxílio da biblioteca *PyTorch* ((Paszke et al., 2019)).

4.3 MÉTODOS PARA LVE

Este estudo realiza a comparação de três diferentes métodos para LVE. Primeiro, o modelo neural generativo adversarial é devidamente treinado em algum conjunto de níveis humanos do jogo *Super Mario Bros*. Após isso, algum dos métodos de evolução de variável latente é aplicado na intenção de guiar a geração de conteúdo para atender propriedades específicas.

Os algoritmos utilizados para a evolução de variável são: o CMA-ES, como em (Volz et al., 2018), o PSO, e o NSGA-II, que foi usado em (Torrado et al., 2019). A motivação para a escolha desses três algoritmos foi, respectivamente, manter o algoritmo utilizado em (Volz et al., 2018) juntamente com seu paradigma (um algoritmo genético), testar a otimização por enxame de partículas por ser simples e rápida, e também avaliar uma abordagem multi-objetivo. A tabela C sintetiza os hiperparâmetros destes três algoritmos.

Para o CMA-ES, a biblioteca utilizada foi a *CMA*((Hansen et al., 2019)). Já para o PSO e o NSGA-II, foram usadas as implementações do *framework* pymoo((Blank e Deb, 2020)).

4.4 FUNÇÕES OBJETIVOS

Neste trabalho, a função objetivo otimizada é relacionada ao número de blocos de um certo tipo do conteúdo gerado, seguindo a equação:

$$\frac{(100 - (\sum_{i=1}^{14} \sum_{j=1}^{28} f(x_{ij}, b)))}{100} \quad \text{tal que} \quad f(x, b) = \begin{cases} 1 & \text{se } x = b \\ 0 & \text{caso contrário} \end{cases}$$

Onde i é o valor de linha e j o da coluna da janela deslizante, e x_{ij} o valor numérico presente nesta posição e b o valor numérico a ser maximizado, ou seja, a representação numérica do bloco a ser máximizad. Os valores de b utilizados foram 6 (um pedaço do cano) e 5 (número de inimigos).

4.5 AVALIAÇÃO DE CONTEÚDO

Como devidamente elicitado por (Hendrikx et al., 2013), uma das maiores dificuldades da PCGML é avaliar o conteúdo gerado proceduralmente. Neste trabalho, foram propostas três métodos de avaliação:

- Os valores das funções objetivos utilizadas na técnica de LVE;
- Um classificador para avaliar se os geradores seguem suficientemente um padrão;
- Um agente para testar a jogabilidade dos níveis.

Para que cada teste deste seja em quantidades suficientes, avaliadas 500 entradas aleatórias e 500 execuções de cada algoritmo para cada função objetivo, com exceção do NSGA-II que, por ser uma abordagem multi-objetivo, otimizou a função para $b = 5$ e $b = 6$ simultâneamente. Na totalidade, por rede, foram avaliados 3000 níveis gerados.

5 RESULTADOS - EM CONSTRUÇÃO, APENAS IMAGENS

5.1 JOGABILIDADE MAXIMIZANDO CANOS

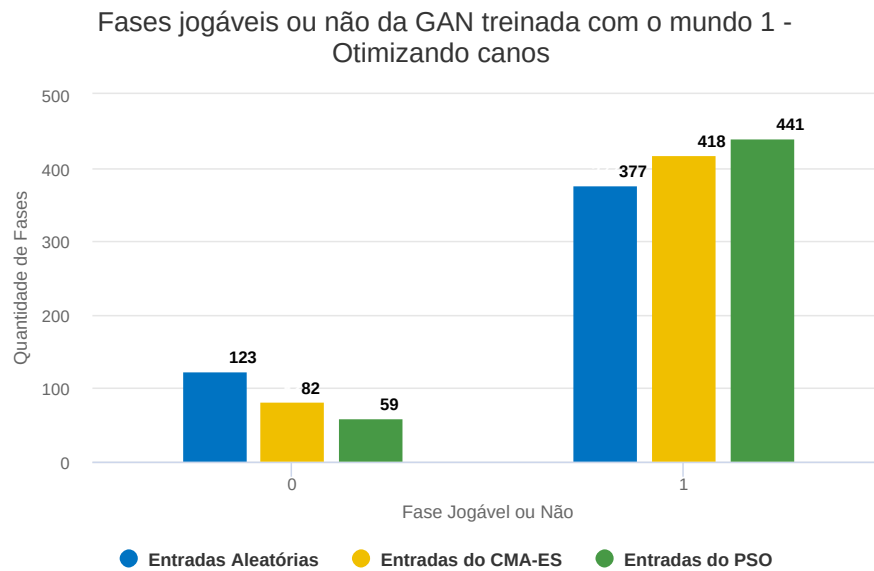


Figura 5.1: Jogabilidade do Mundo 1

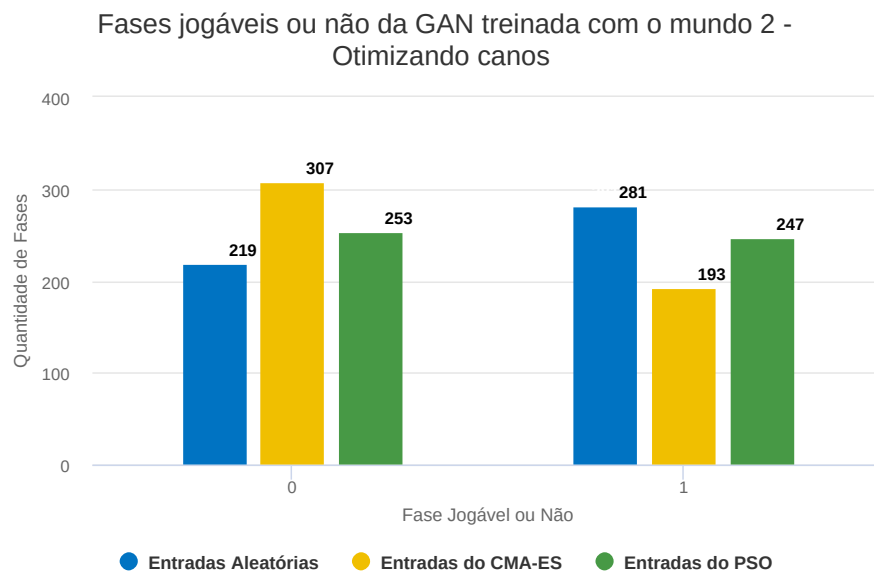


Figura 5.2: Jogabilidade do Mundo 2

Mundo 8:

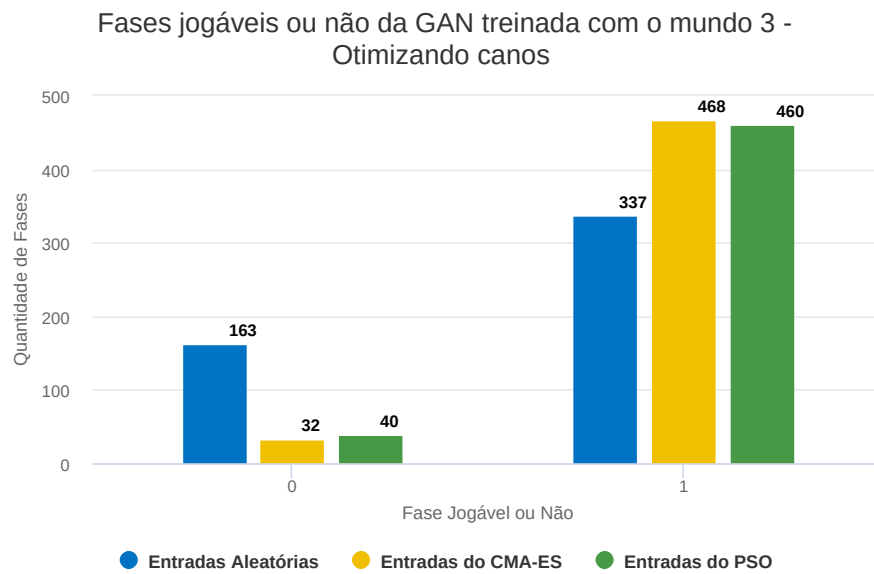


Figura 5.3: Jogabilidade do Mundo 3

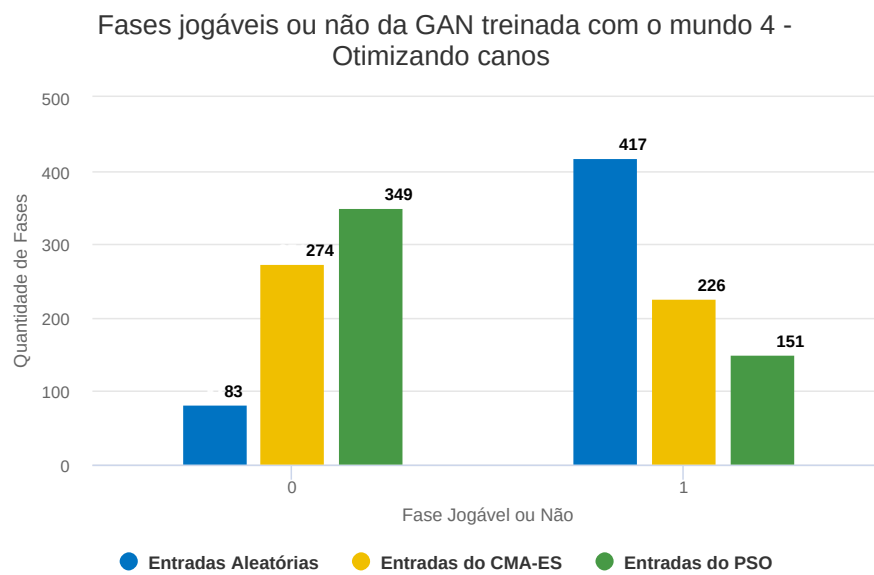


Figura 5.4: Jogabilidade do Mundo 4

5.2 JOGABILIDADE MAXIMIZANDO INIMIGOS

Mundo 1:

Jogabilidade Mundo 2:

Mundo 3:

Mundo 4:

Mundo 5:

Mundo 6:

Mundo 7:

Mundo 8:

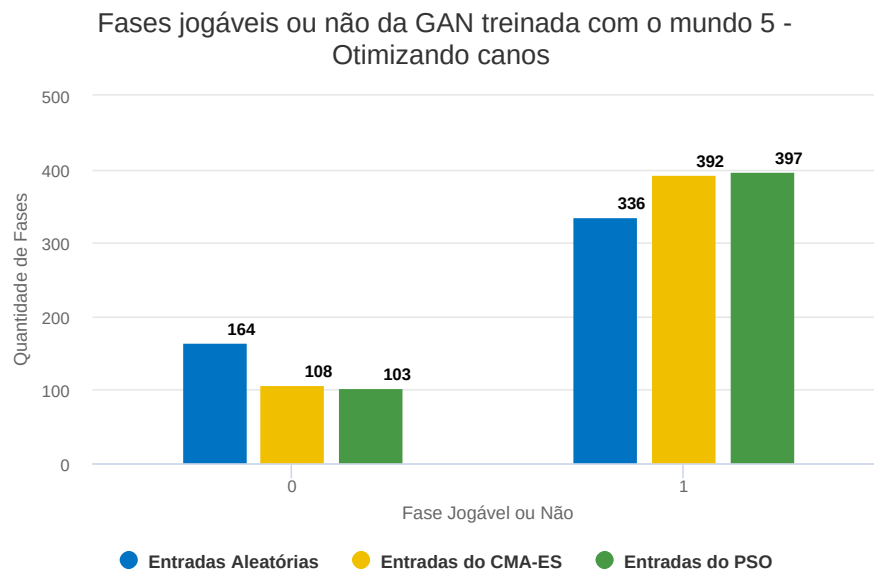


Figura 5.5: Jogabilidade do Mundo 5

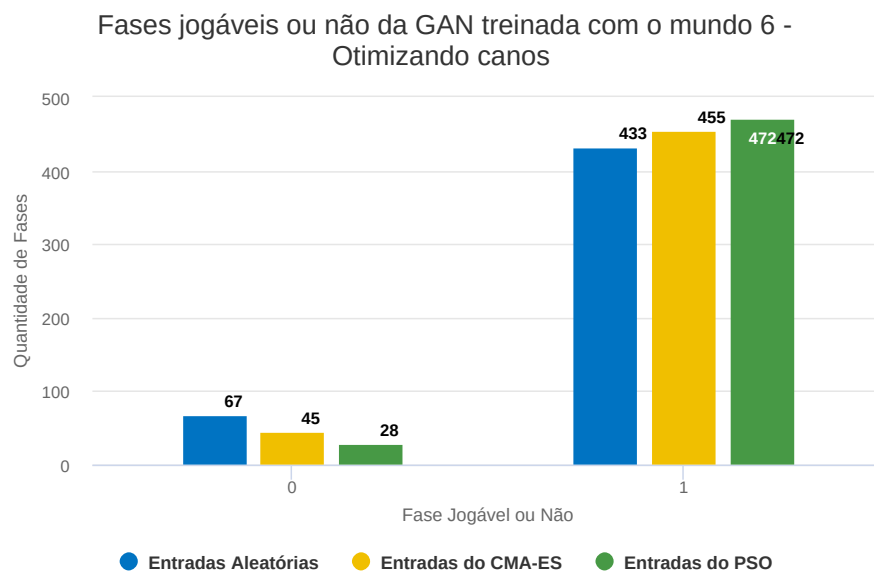


Figura 5.6: Jogabilidade do Mundo 6

5.3 TABELA DE VALORES OTIMIZANDO CANOS

Tabela de valores otimizando canos:

5.4 TABELA DE VALORES OTIMIZANDO INIMIGOS

Tabela:

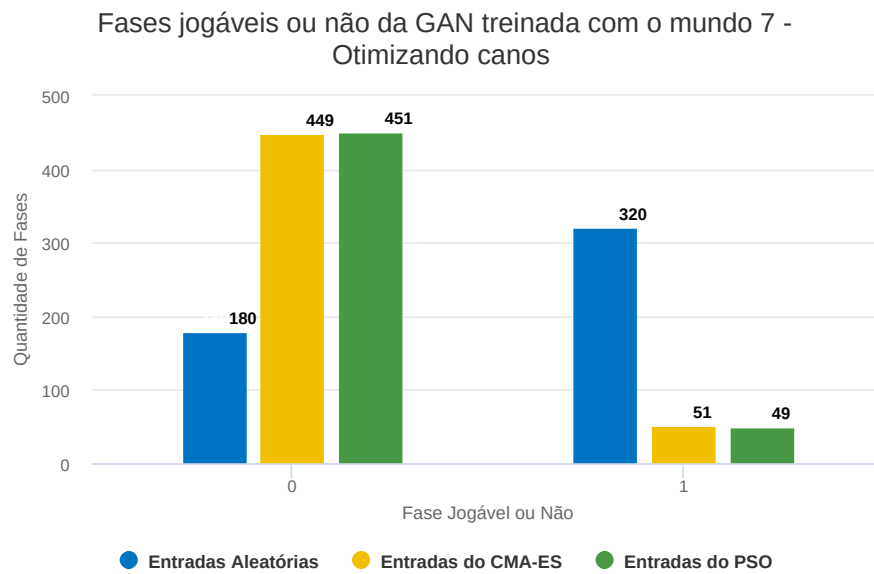


Figura 5.7: Jogabilidade do Mundo 7

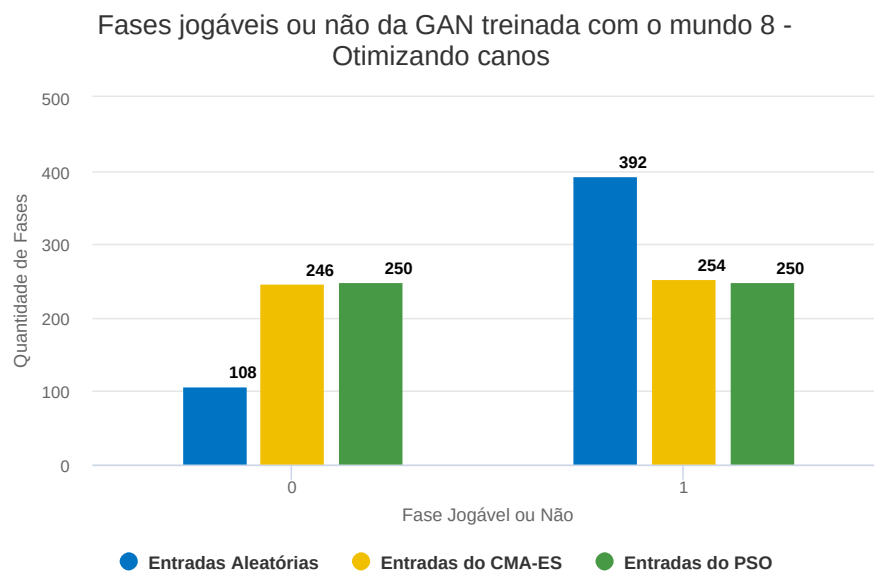


Figura 5.8: Jogabilidade do Mundo 8

5.5 CLASSIFICAÇÃO DE PADRÕES

Primeiro Mundo
 Segundo Mundo
 Terceiro Mundo
 Quarto Mundo
 Quinto Mundo
 Sexto Mundo
 Sétimo Mundo
 Oitavo Mundo

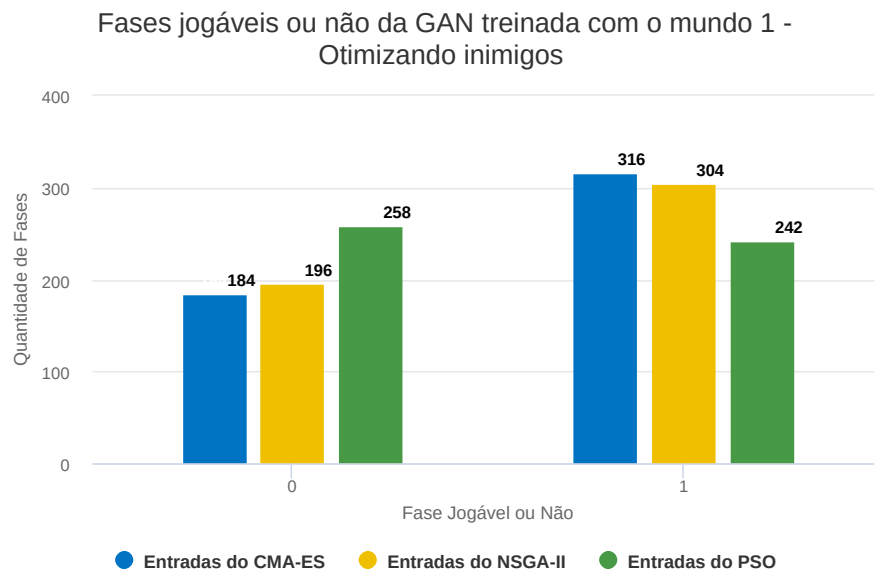


Figura 5.9: Jogabilidade do Mundo 1

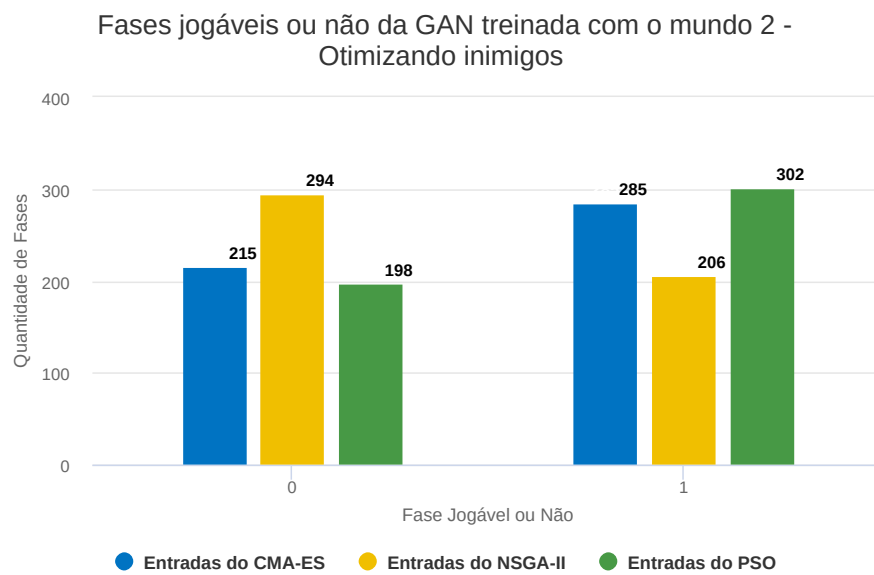


Figura 5.10: Jogabilidade do Mundo 2

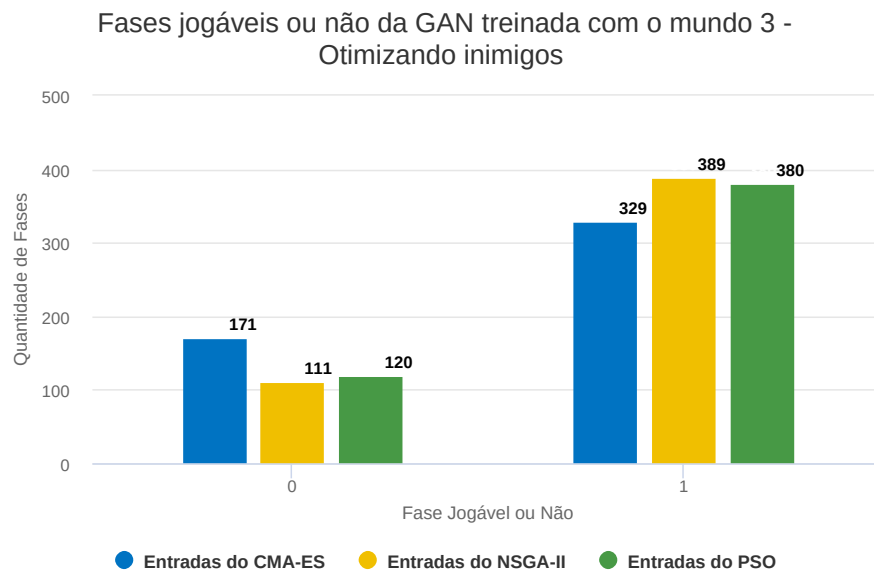


Figura 5.11: Jogabilidade do Mundo 3

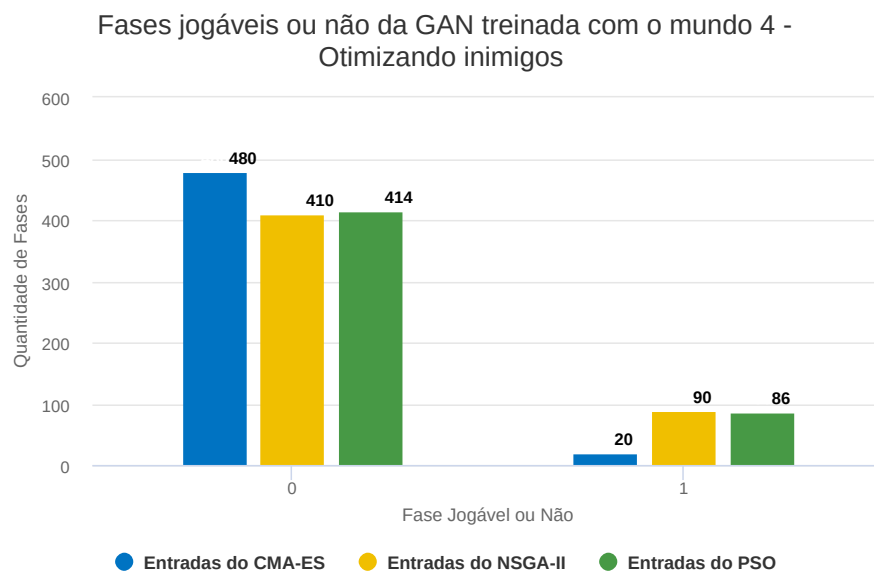


Figura 5.12: Jogabilidade do Mundo 4

5.6 NÍVEIS

5.6.1 Mundo 1

5.6.2 Mundo 2

5.6.3 Mundo 3

5.6.4 Mundo 4

5.6.5 Mundo 5

5.6.6 Mundo 6

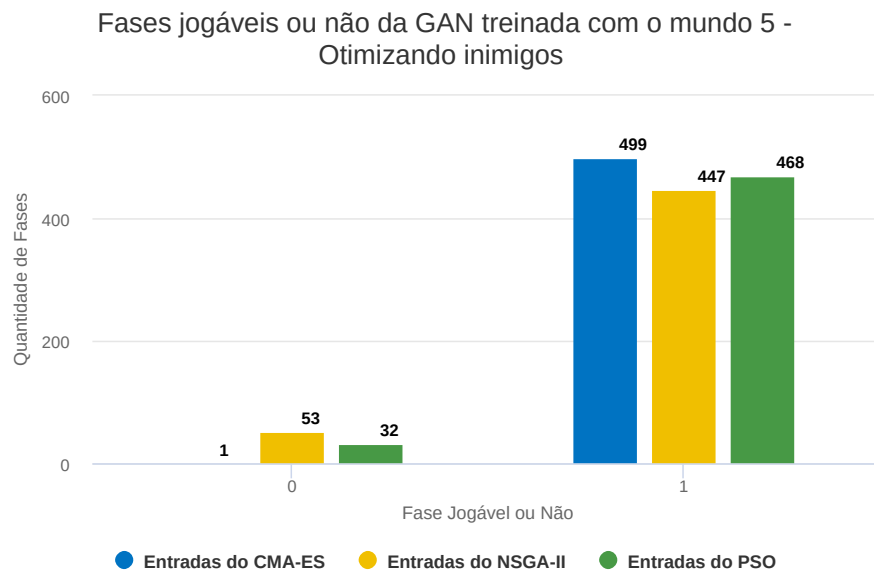


Figura 5.13: Jogabilidade do Mundo 5

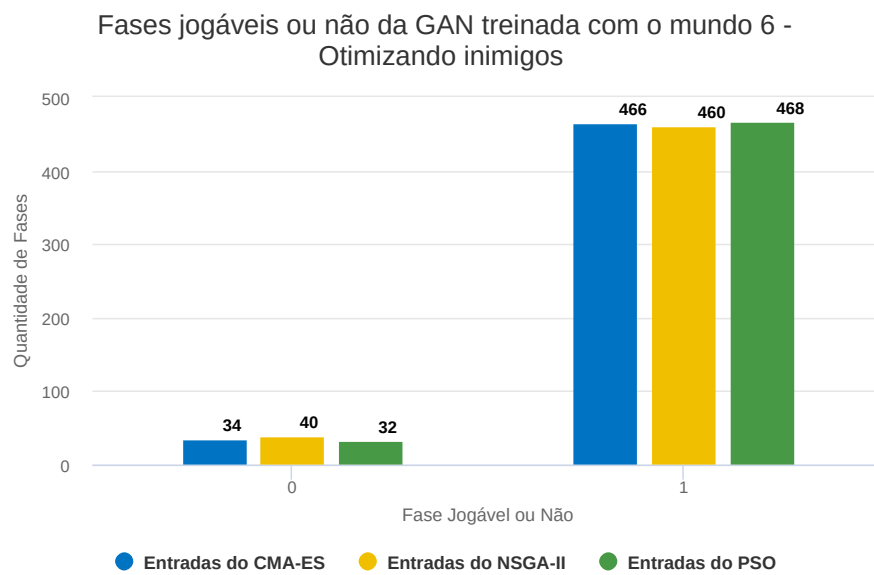


Figura 5.14: Jogabilidade do Mundo 6

=

==>

=

>>

5.6.7 Mundo 7

òóò>
đóñđ

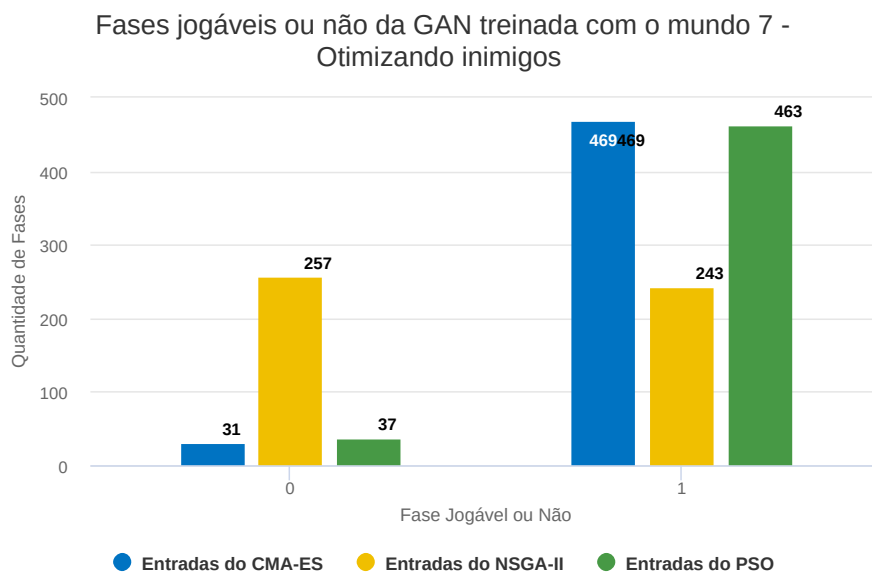


Figura 5.15: Jogabilidade do Mundo 7

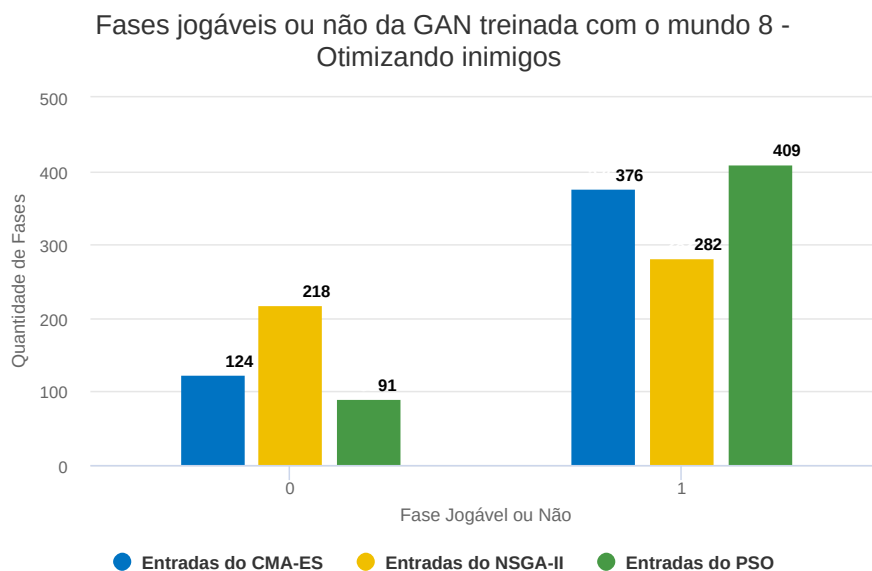


Figura 5.16: Jogabilidade do Mundo 8

entrada	Mean Canos	Max canos	Min canos	Median Canos	Std canos	Mean Inimigos	Max Inimigos	Min Inimigos	Median Inimigos	Std Inimigos	mundo
1 Entradas Aleatórias	0.995	1	0.97	1	0.00715	0.982	1	0.91	0.99	0.0157	Mundo 1
2 Entradas Aleatórias	0.994	1	0.98	0.99	0.00548	0.959	1	0.89	0.96	0.0268	Mundo 2
3 Entradas Aleatórias	0.997	1	0.97	1	0.00512	0.977	1	0.91	0.98	0.0183	Mundo 3
4 Entradas Aleatórias	0.993	1	0.96	1	0.00832	0.991	1	0.95	0.99	0.0185	Mundo 4
5 Entradas Aleatórias	0.998	1	0.97	1	0.00527	0.969	1	0.89	0.97	0.0214	Mundo 5
6 Entradas Aleatórias	0.999	1	0.98	1	0.00293	0.999	1	0.98	1	0.00312	Mundo 6
7 Entradas Aleatórias	0.995	1	0.96	1	0.00733	0.987	1	0.95	0.99	0.0114	Mundo 7
8 Entradas Aleatórias	0.992	1	0.95	0.99	0.00922	0.964	1	0.9	0.97	0.0196	Mundo 8
9 Entradas do CMA-ES	0.942	0.96	0.94	0.94	0.00558	0.988	1	0.96	0.99	0.00522	Mundo 1
10 Entradas do CMA-ES	0.966	0.99	0.94	0.97	0.0112	0.954	1	0.9	0.95	0.0226	Mundo 2
11 Entradas do CMA-ES	0.971	0.98	0.97	0.97	0.00365	0.961	1	0.91	0.96	0.0151	Mundo 3
12 Entradas do CMA-ES	0.943	0.95	0.94	0.94	0.00467	0.956	0.98	0.92	0.96	0.0182	Mundo 4
13 Entradas do CMA-ES	0.959	1	0.95	0.96	0.00599	0.966	1	0.91	0.97	0.0138	Mundo 5
14 Entradas do CMA-ES	0.982	1	0.97	0.98	0.00543	0.987	1	0.98	0.99	0.00724	Mundo 6
15 Entradas do CMA-ES	0.952	0.97	0.93	0.95	0.00488	0.975	0.99	0.94	0.98	0.00728	Mundo 7
16 Entradas do CMA-ES	0.942	0.96	0.93	0.94	0.00398	0.961	0.99	0.93	0.96	0.00878	Mundo 8
17 Entradas do PSO	0.954	0.97	0.94	0.95	0.00934	0.985	1	0.94	0.99	0.0186	Mundo 1
18 Entradas do PSO	0.968	0.98	0.95	0.97	0.00882	0.953	1	0.88	0.95	0.0232	Mundo 2
19 Entradas do PSO	0.972	0.98	0.97	0.97	0.00383	0.961	1	0.92	0.96	0.0162	Mundo 3
20 Entradas do PSO	0.947	0.97	0.94	0.95	0.00438	0.956	0.99	0.93	0.96	0.0191	Mundo 4
21 Entradas do PSO	0.963	0.99	0.95	0.96	0.00637	0.965	1	0.93	0.96	0.0126	Mundo 5
22 Entradas do PSO	0.980	1	0.98	0.98	0.00899	0.984	1	0.98	0.98	0.00612	Mundo 6
23 Entradas do PSO	0.954	0.97	0.93	0.95	0.00705	0.975	0.99	0.95	0.98	0.00758	Mundo 7
24 Entradas do PSO	0.948	0.97	0.93	0.95	0.00702	0.961	0.99	0.89	0.96	0.0137	Mundo 8

Figura 5.17: Tabela Maximizando canos

entrada	Mean canos	Max canos	Min canos	Median canos	Std canos	Mean inimigos	Max inimigos	Min inimigos	Median inimigos	Std inimigos	Mundo
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1 Entradas do CWA-ES	0.999	1	0.98	1	0.00391	0.890	0.93	0.86	0.89	0.00793	Mundo 1
2 Entradas do CWA-ES	0.994	1	0.98	0.99	0.00536	0.847	0.88	0.8	0.85	0.0122	Mundo 2
3 Entradas do CWA-ES	0.992	1	0.98	0.99	0.00512	0.878	0.91	0.85	0.88	0.0104	Mundo 3
4 Entradas do CWA-ES	0.967	1	0.94	0.97	0.00844	0.930	0.94	0.92	0.93	0.000725	Mundo 4
5 Entradas do CWA-ES	1	1	1	1	0	0.848	0.88	0.84	0.84	0.0116	Mundo 5
6 Entradas do CWA-ES	0.987	1	0.98	0.99	0.00673	0.981	1	0.97	0.98	0.00628	Mundo 6
7 Entradas do CWA-ES	0.985	1	0.96	0.98	0.00647	0.932	0.96	0.92	0.93	0.00564	Mundo 7
8 Entradas do CWA-ES	0.989	1	0.97	0.99	0.00917	0.836	0.88	0.82	0.83	0.00921	Mundo 8
9 Entradas do NSGA-II	0.986	1	0.97	0.99	0.00807	0.940	1	0.89	0.94	0.0185	Mundo 1
10 Entradas do NSGA-II	0.998	1	0.97	0.99	0.00458	0.907	0.97	0.85	0.91	0.0178	Mundo 2
11 Entradas do NSGA-II	0.985	1	0.97	0.99	0.00686	0.928	0.99	0.89	0.93	0.0183	Mundo 3
12 Entradas do NSGA-II	0.968	1	0.95	0.96	0.00778	0.948	0.99	0.93	0.95	0.0101	Mundo 4
13 Entradas do NSGA-II	0.995	1	0.97	1	0.00802	0.893	0.97	0.84	0.89	0.0221	Mundo 5
14 Entradas do NSGA-II	0.987	1	0.98	0.99	0.00536	0.987	1	0.98	0.99	0.00547	Mundo 6
15 Entradas do NSGA-II	0.976	1	0.95	0.98	0.01102	0.955	0.99	0.92	0.95	0.0117	Mundo 7
16 Entradas do NSGA-II	0.984	1	0.96	0.98	0.0101	0.901	0.98	0.84	0.9	0.0205	Mundo 8
17 Entradas do PSO	0.998	1	0.98	1	0.00442	0.898	0.94	0.87	0.9	0.0121	Mundo 1
18 Entradas do PSO	0.993	1	0.98	0.99	0.00574	0.862	0.89	0.82	0.86	0.0115	Mundo 2
19 Entradas do PSO	0.993	1	0.98	0.99	0.00585	0.869	0.92	0.85	0.89	0.0115	Mundo 3
20 Entradas do PSO	0.970	1	0.95	0.97	0.01102	0.932	0.95	0.92	0.93	0.00445	Mundo 4
21 Entradas do PSO	1.00	1	0.98	1	0.00270	0.865	0.9	0.84	0.87	0.0152	Mundo 5
22 Entradas do PSO	0.986	1	0.98	0.99	0.00556	0.979	0.98	0.97	0.98	0.00230	Mundo 6
23 Entradas do PSO	0.989	1	0.97	0.99	0.00607	0.936	0.96	0.92	0.94	0.00631	Mundo 7
24 Entradas do PSO	0.991	1	0.97	0.99	0.00977	0.857	0.91	0.83	0.85	0.0160	Mundo 8

Figura 5.18: Tabela Inimigos

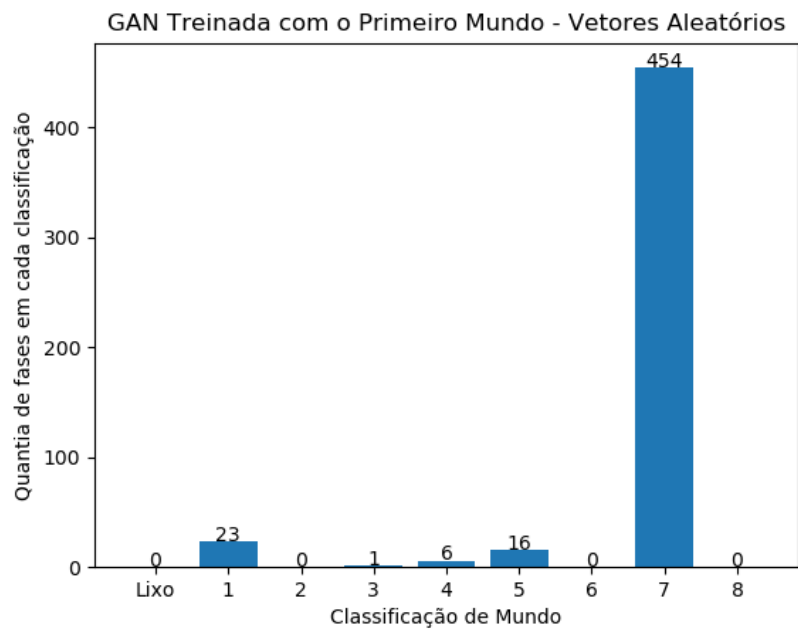


Figura 5.19: Primeiro Mundo

ðïïóñðïï

ðïïóñðïïñë

5.6.8 Mundo 8

ðïïñëóñðïïñë

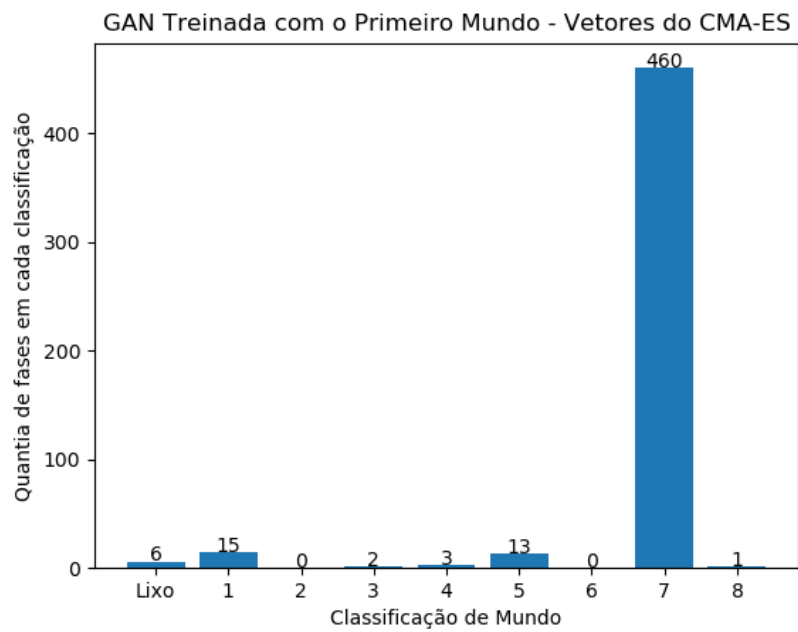


Figura 5.20: Primeiro Mundo

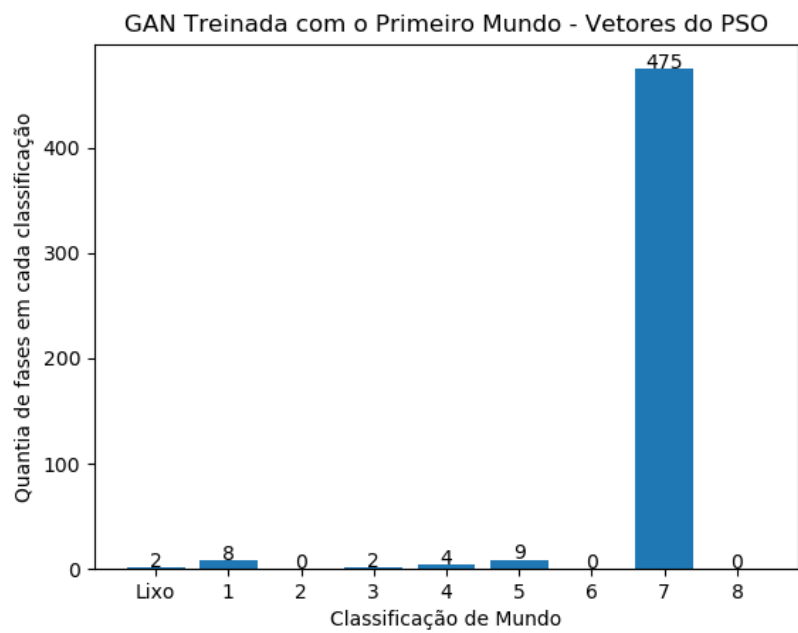


Figura 5.21: Primeiro Mundo

öïïëöñöïïë

öïïëöñöïïëêé

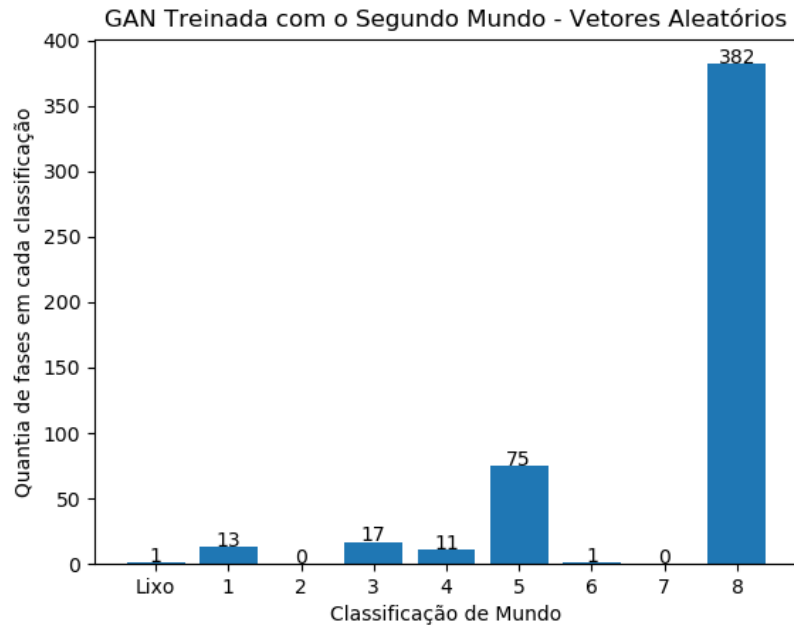


Figura 5.22: segundo Mundo

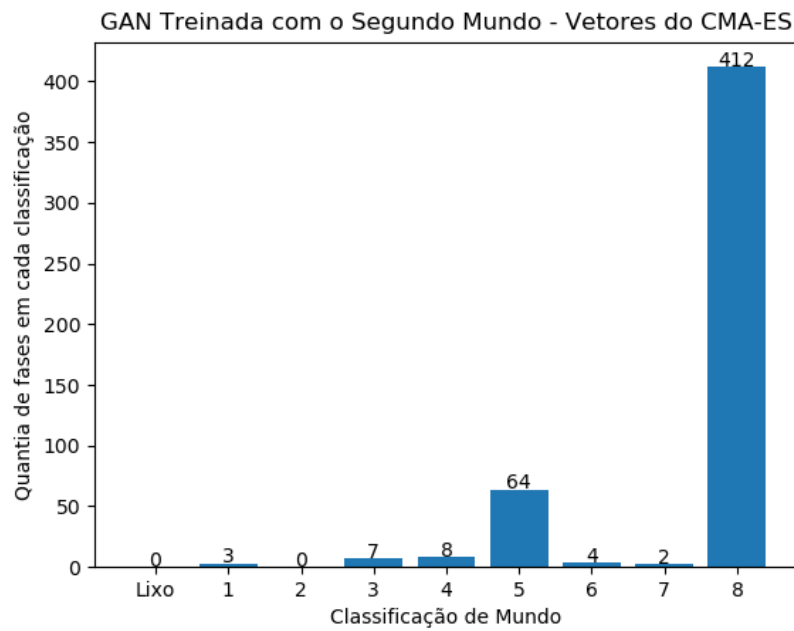


Figura 5.23: Segundo Mundo

ðïñëëóñðïñëë

ðïñëëóñðïñëëç

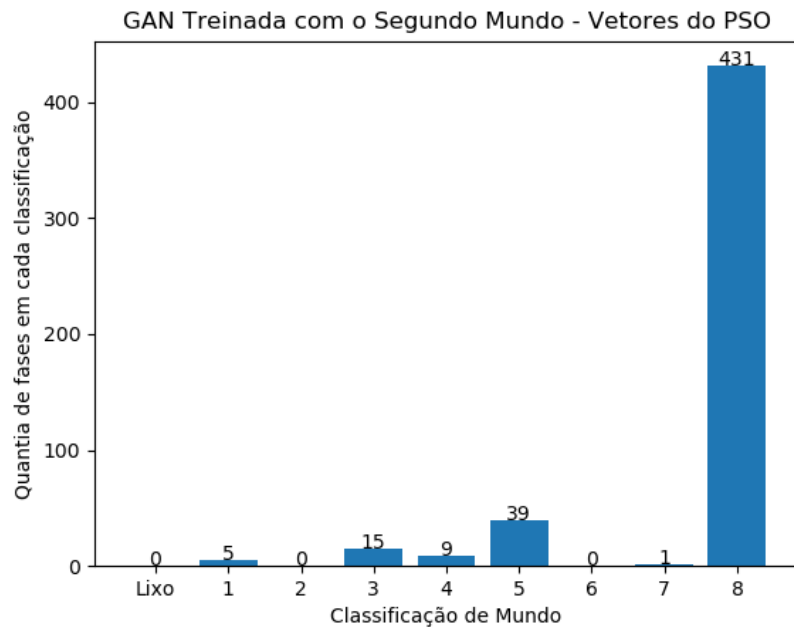


Figura 5.24: Segundo Mundo

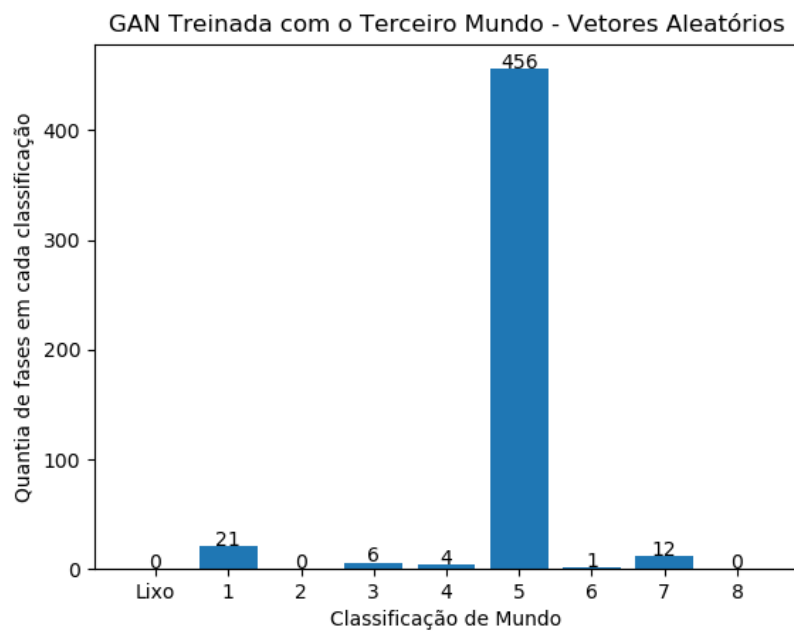


Figura 5.25: 39 Mundo

ðïñëëëëçónðïñëëëëç

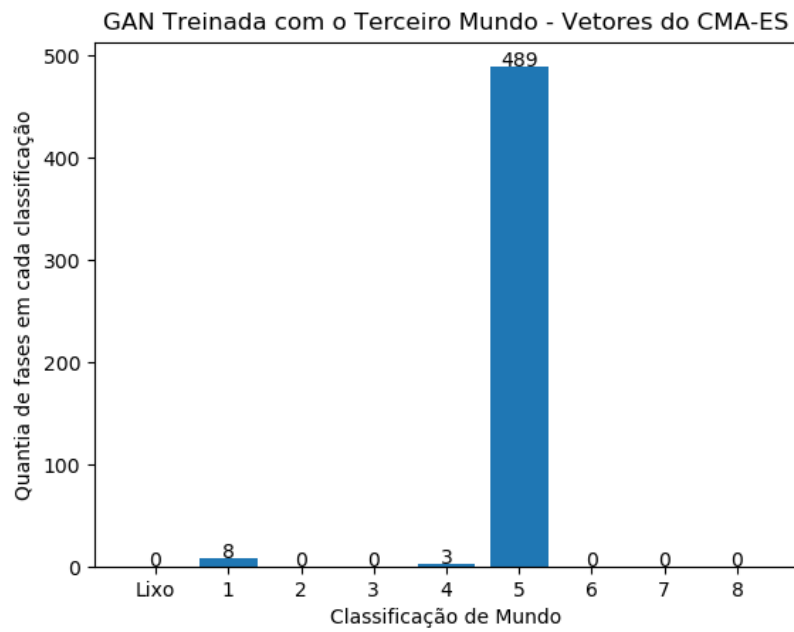


Figura 5.26: 3o Mundo

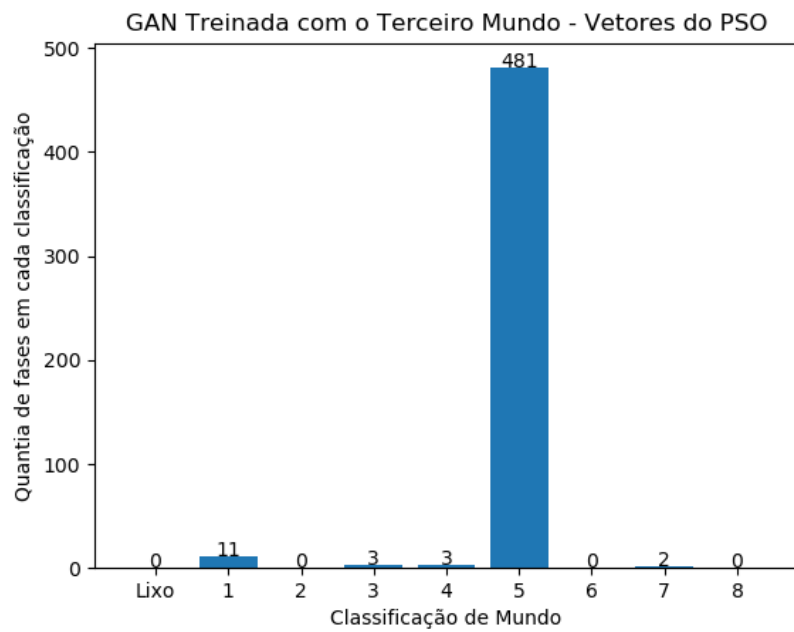


Figura 5.27: 3o Mundo

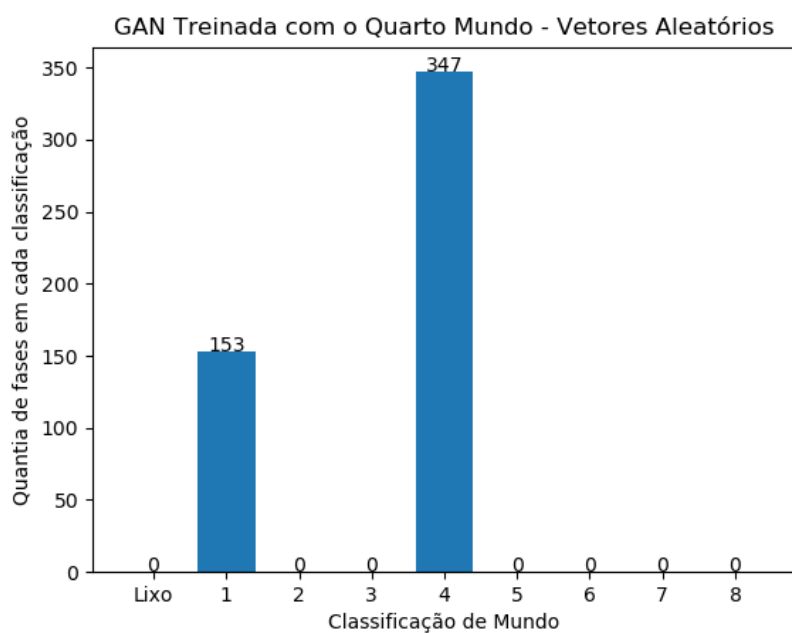


Figura 5.28: 4o Mundo

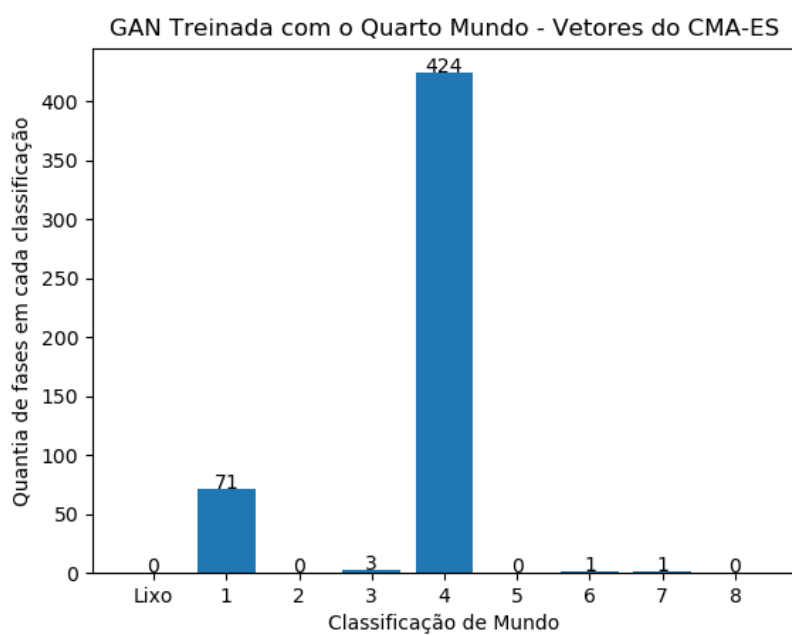


Figura 5.29: 4o Mundo

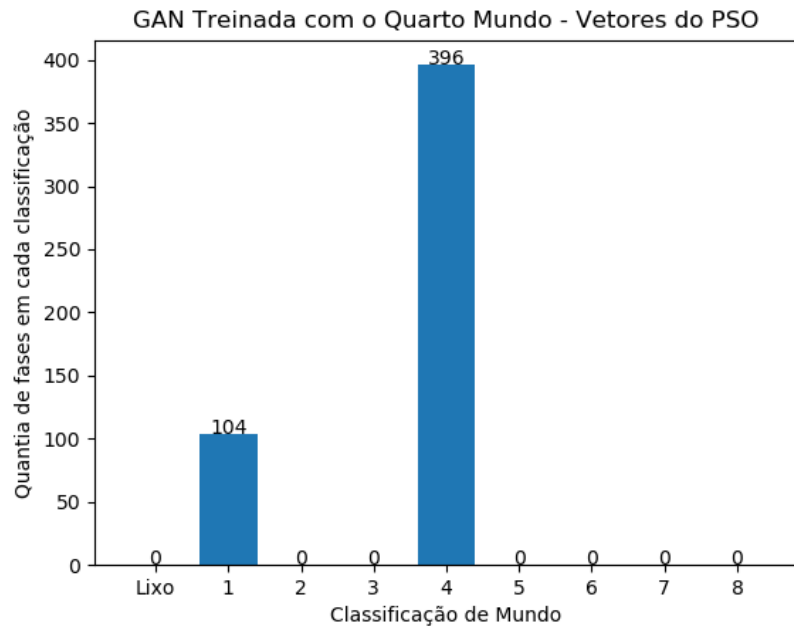


Figura 5.30: 4o Mundo

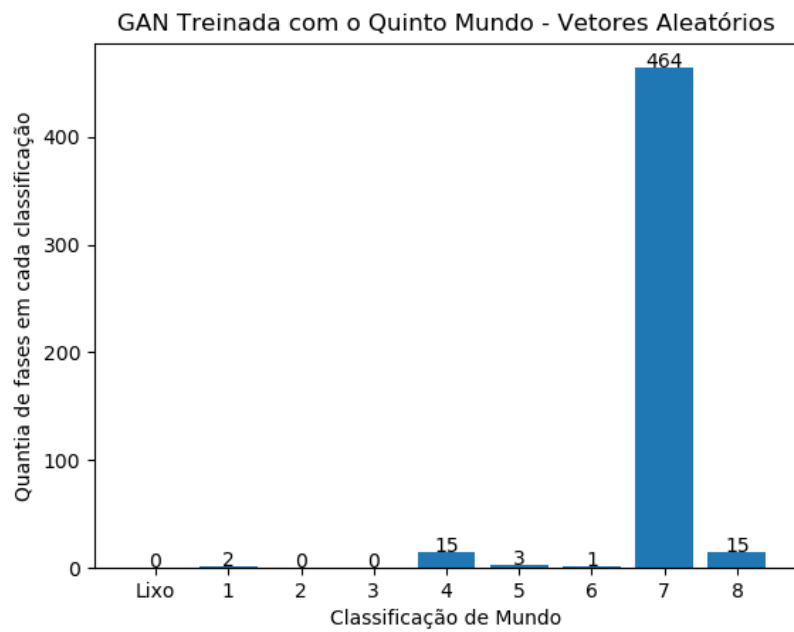


Figura 5.31: 5 Mundo

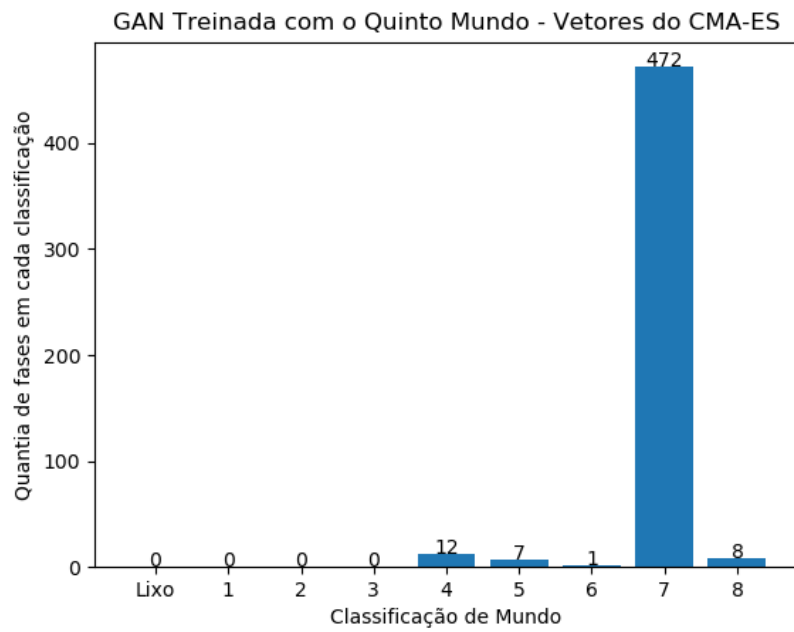


Figura 5.32: 5o Mundo

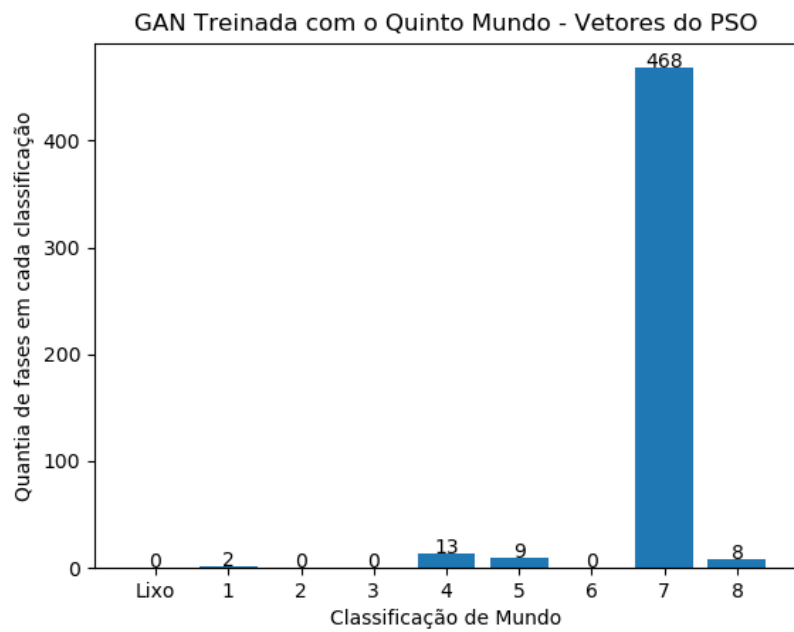


Figura 5.33: 5o Mundo

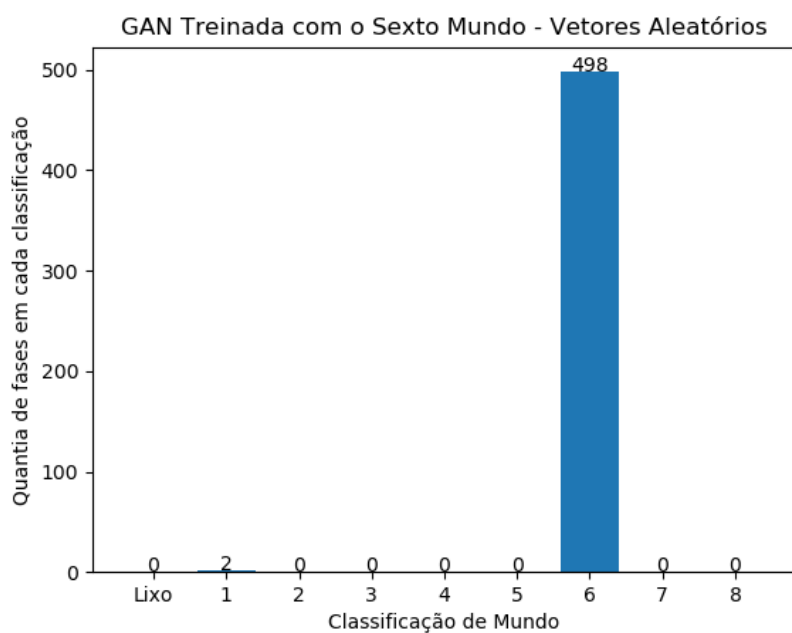


Figura 5.34: 6 Mundo

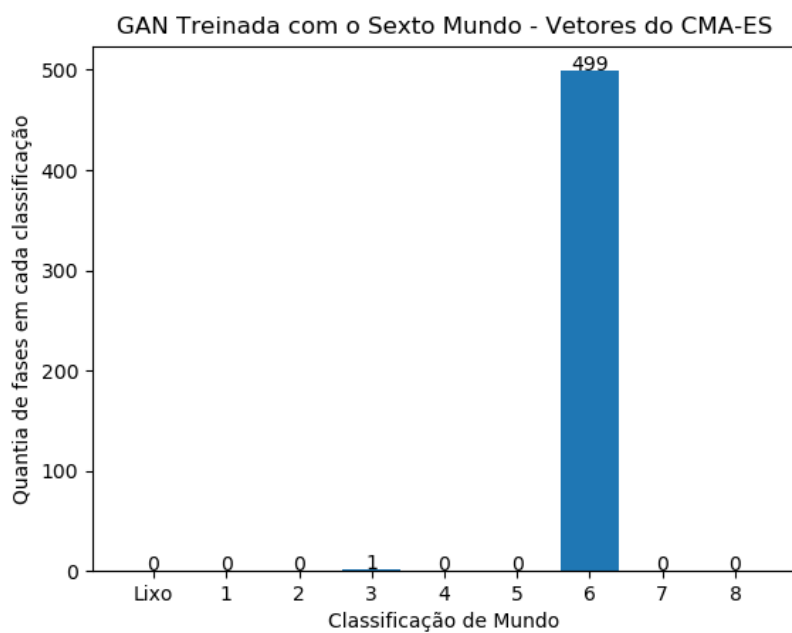


Figura 5.35: 6o Mundo

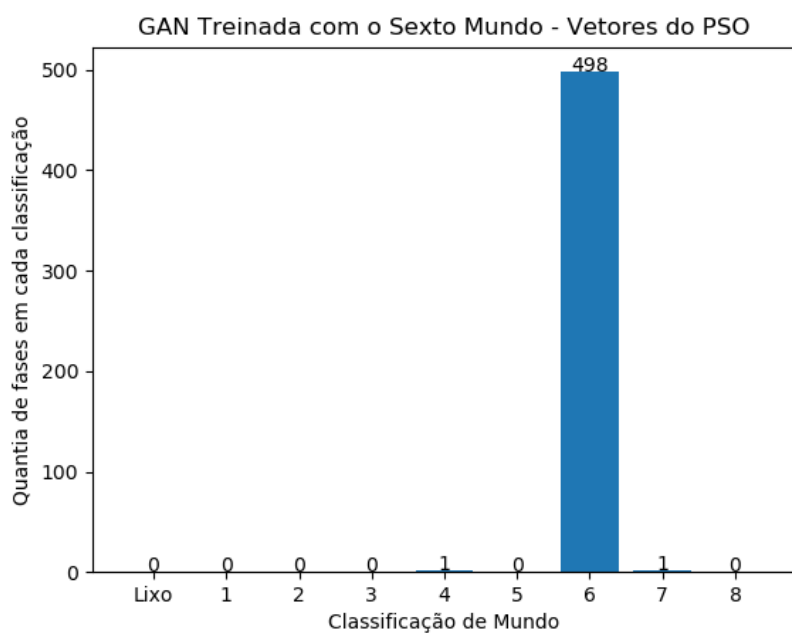


Figura 5.36: 6o Mundo

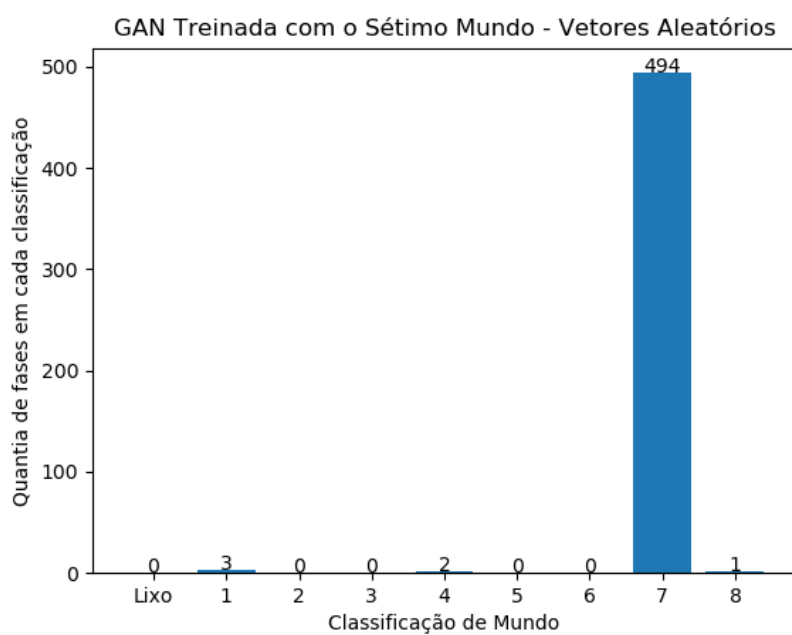


Figura 5.37: 7 Mundo

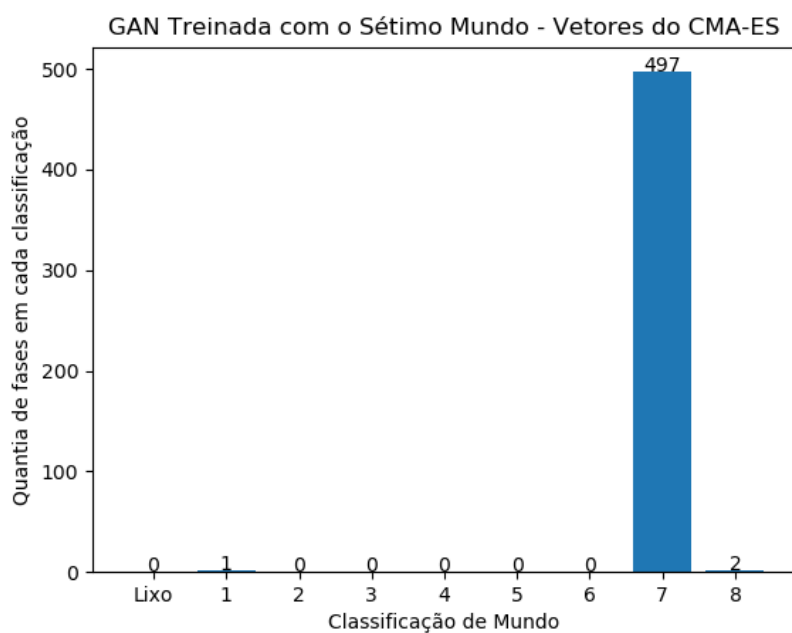


Figura 5.38: 7o Mundo

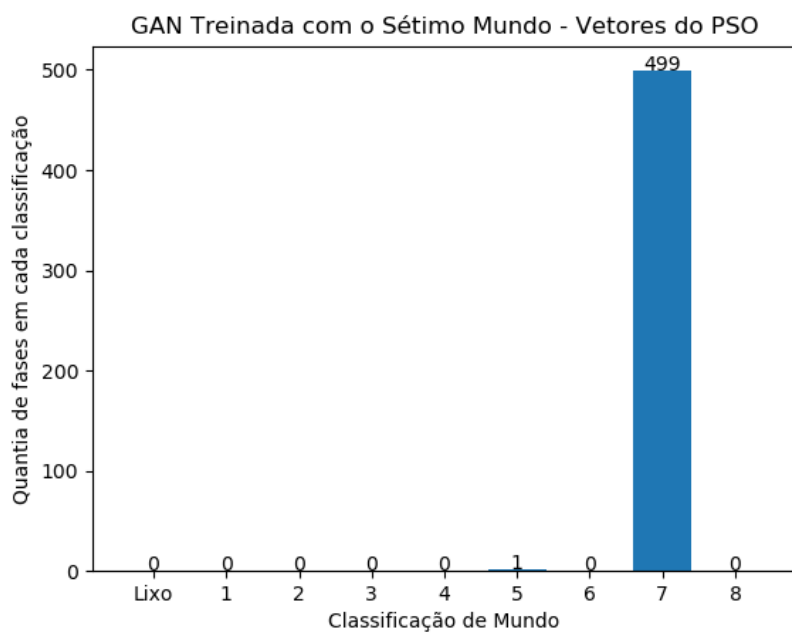


Figura 5.39: 7o Mundo

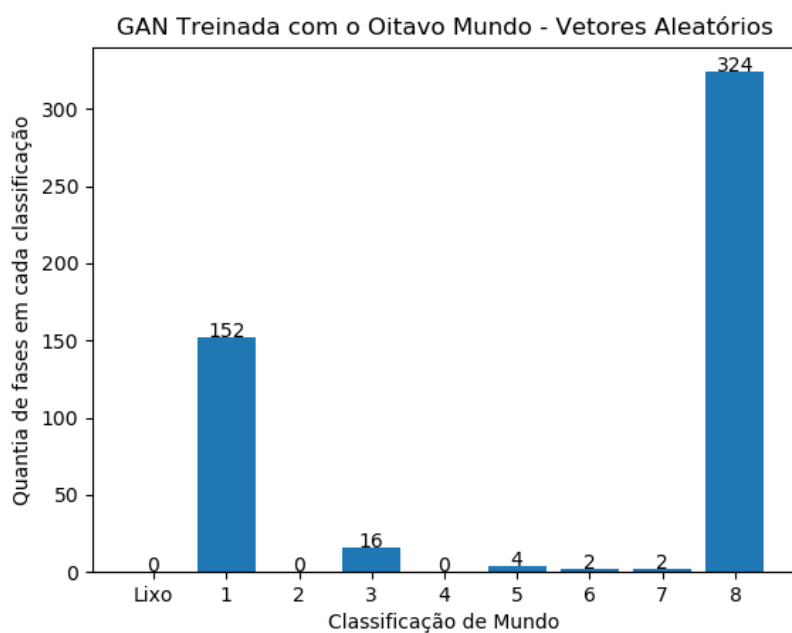


Figura 5.40: 8 Mundo

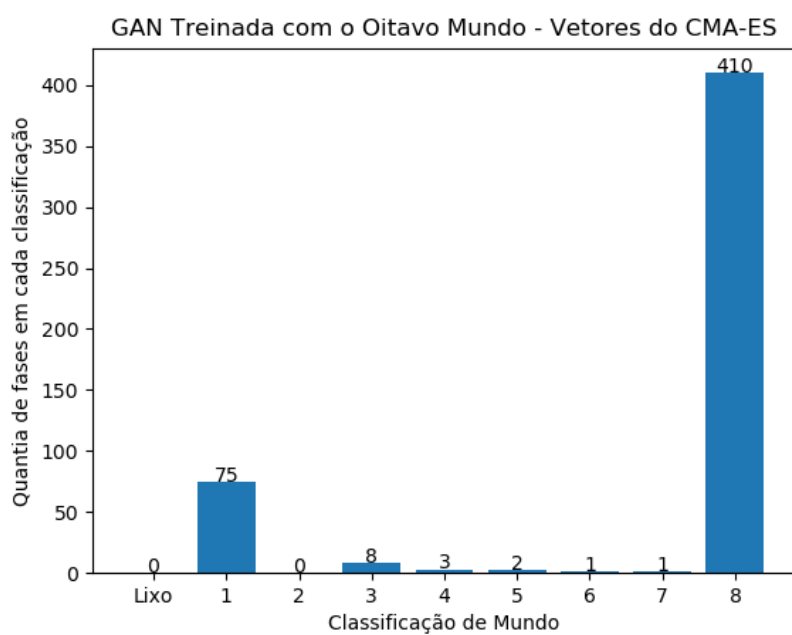


Figura 5.41: 8o Mundo

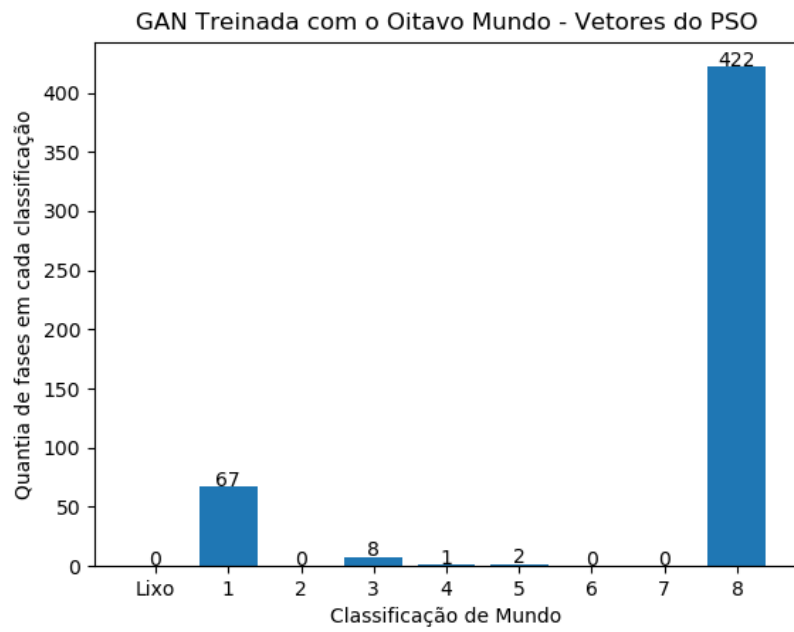


Figura 5.42: 8o Mundo

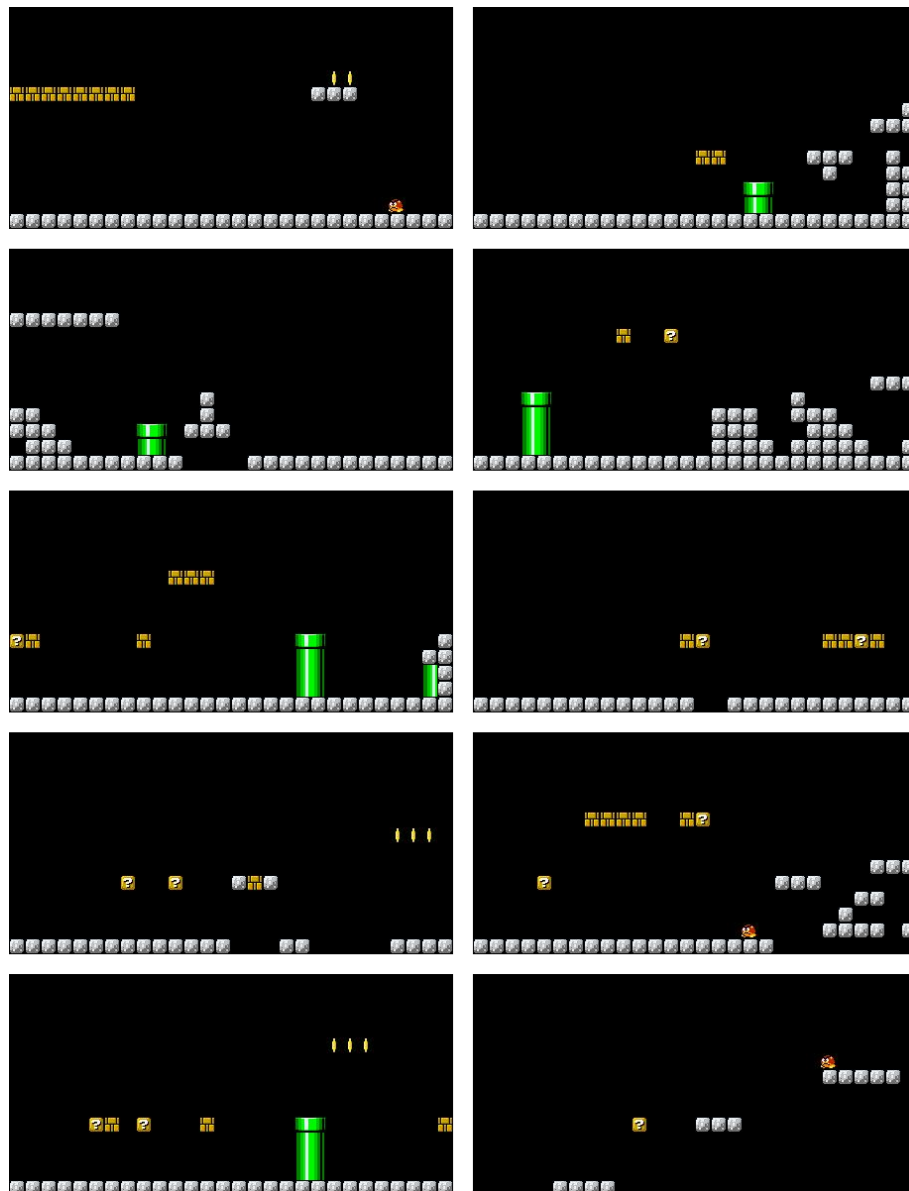


Figura 5.43: 1º Mundo - Níveis Aleatórios

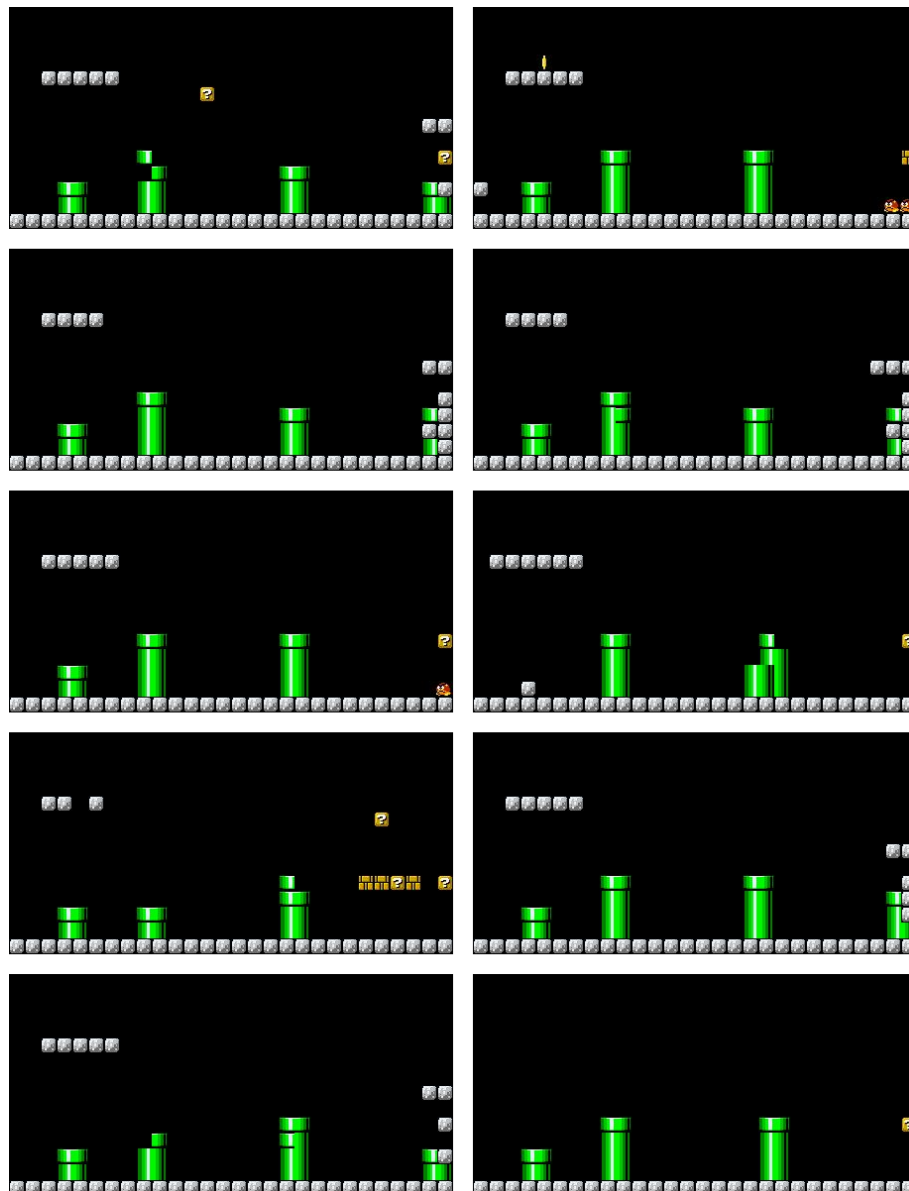


Figura 5.44: 1º Mundo - Níveis Do CMA-ES - Otimizando Canos

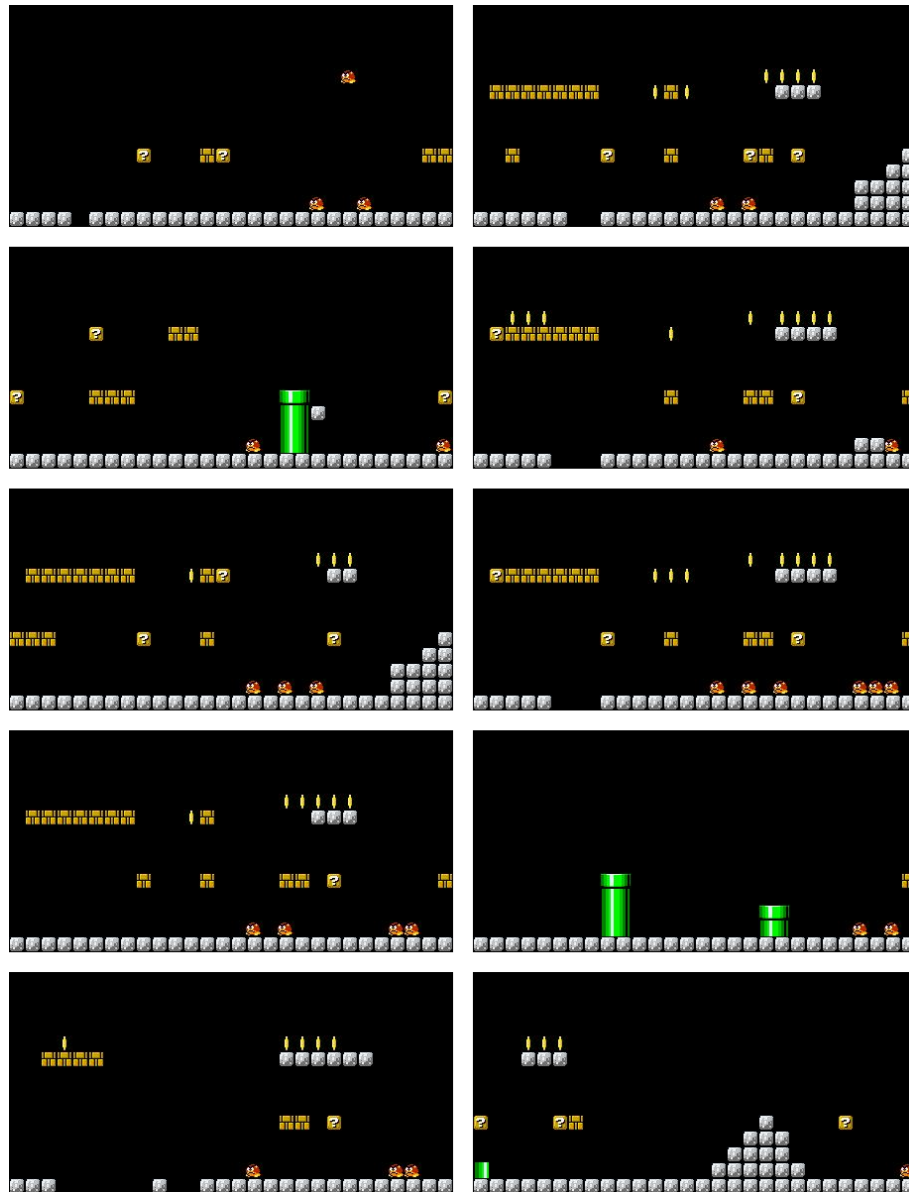


Figura 5.45: 1º Mundo - Níveis Do CMA-ES - Otimizando Inimigos

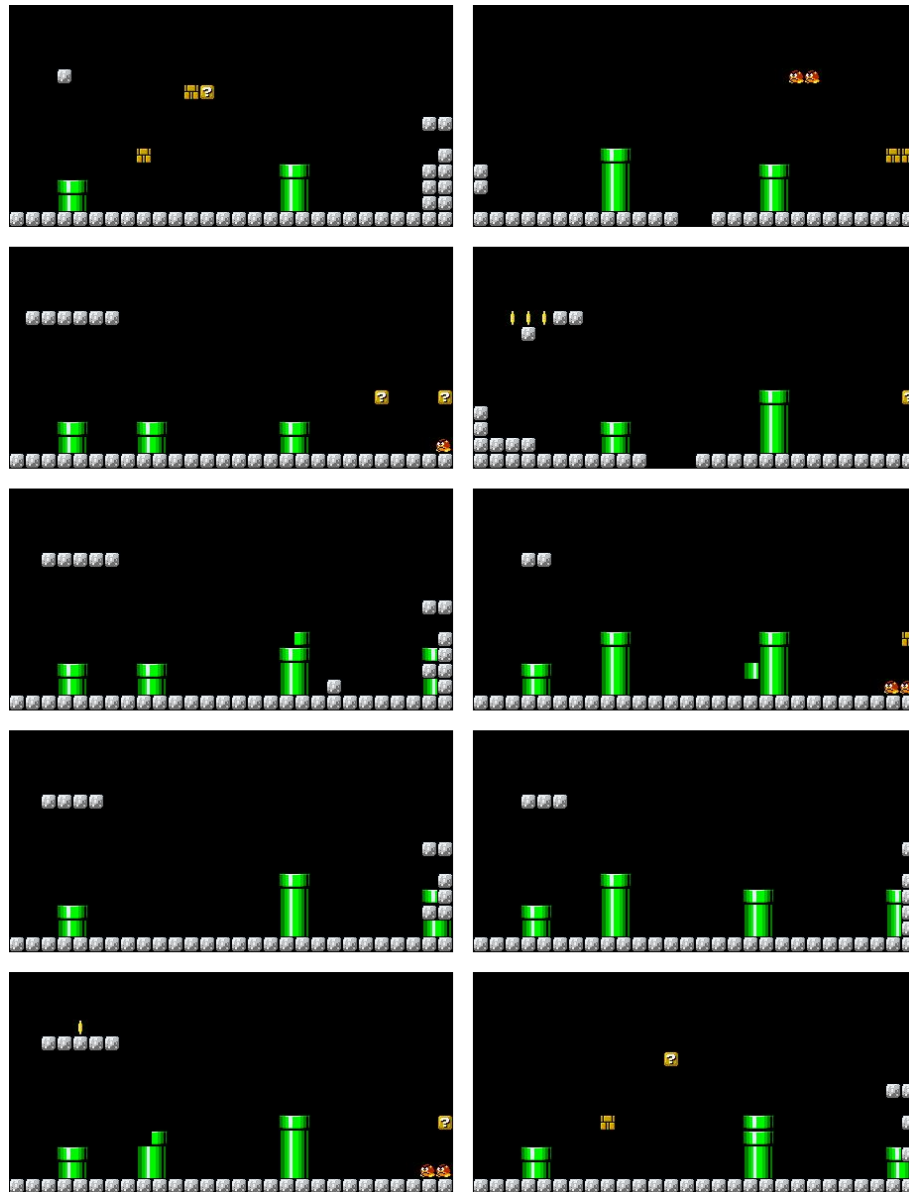


Figura 5.46: 1º Mundo - Níveis Do PSO - Otimizando Canos

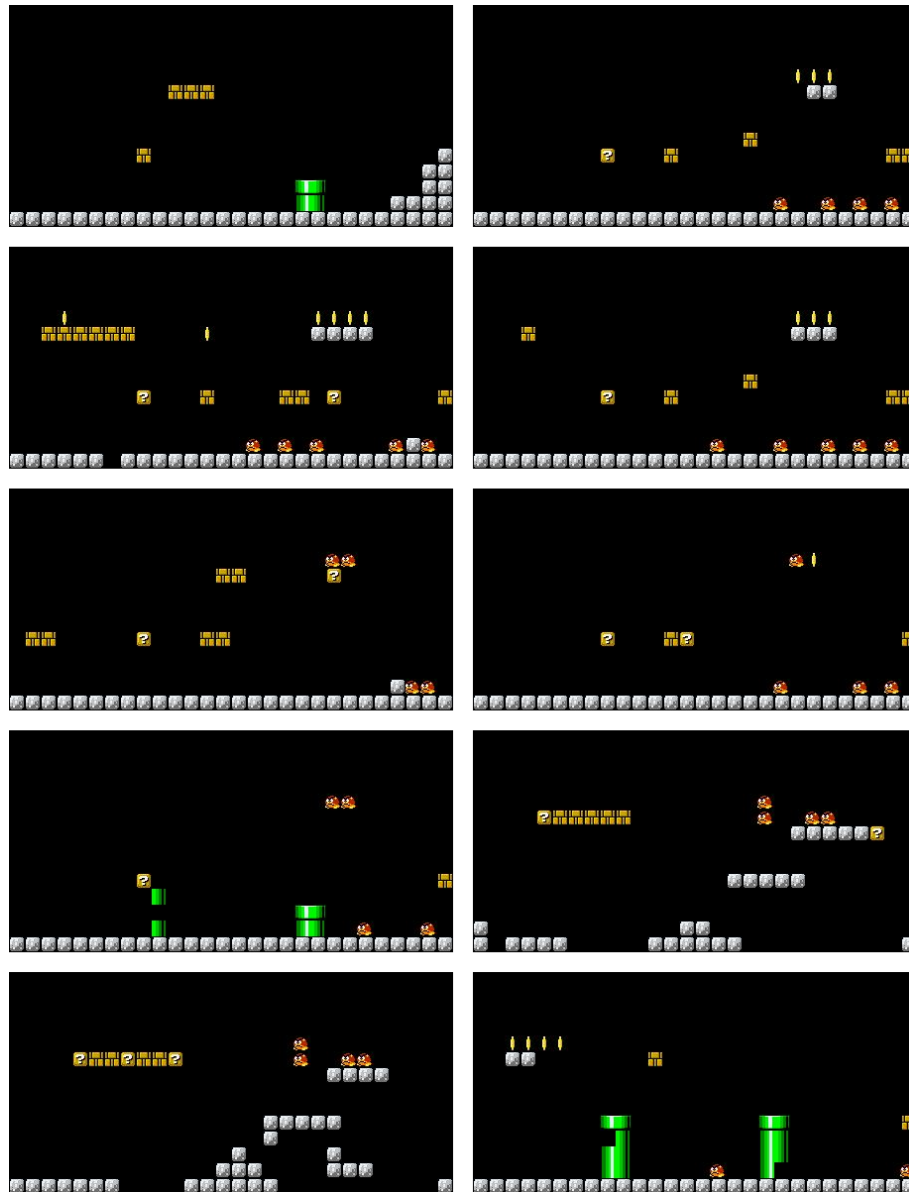


Figura 5.47: 1º Mundo - Níveis Do PSO - Otimizando Inimigos

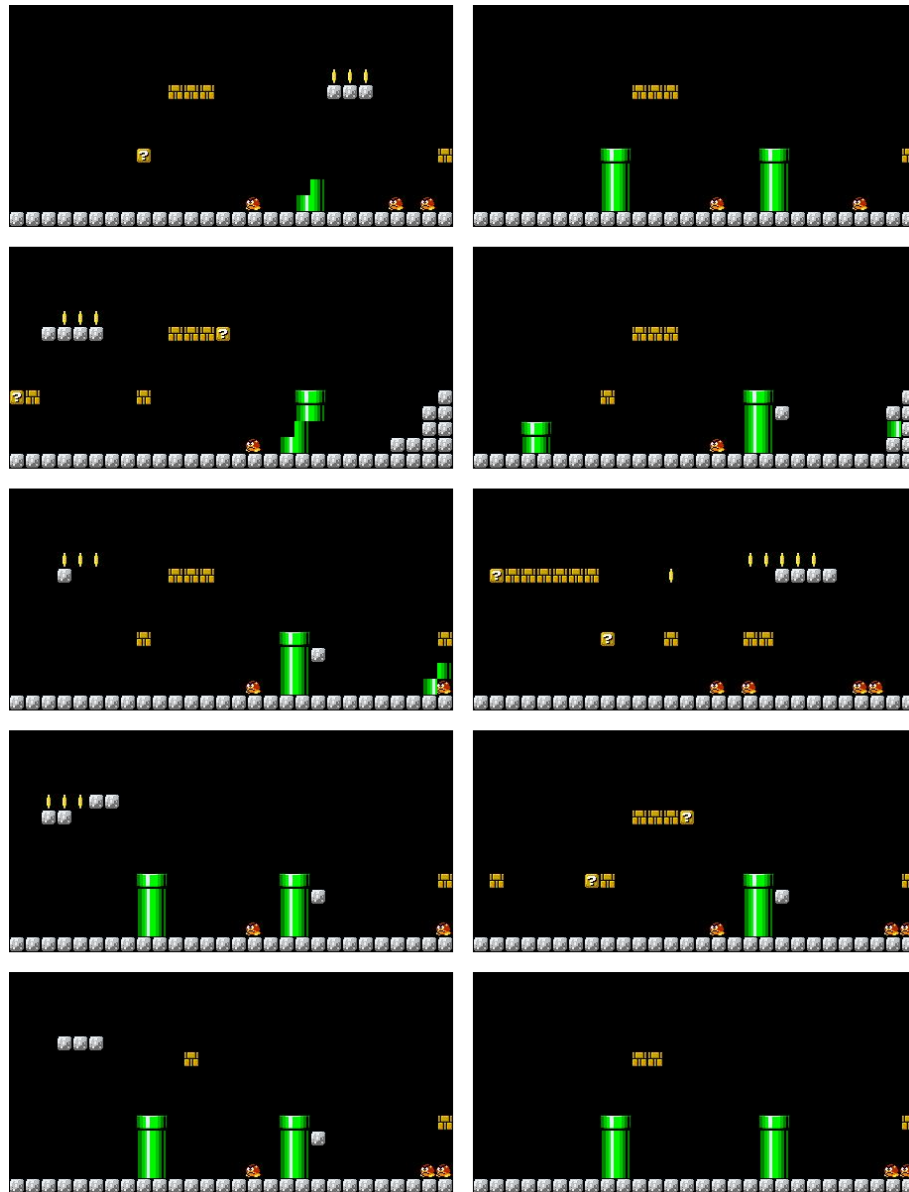


Figura 5.48: 1º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos

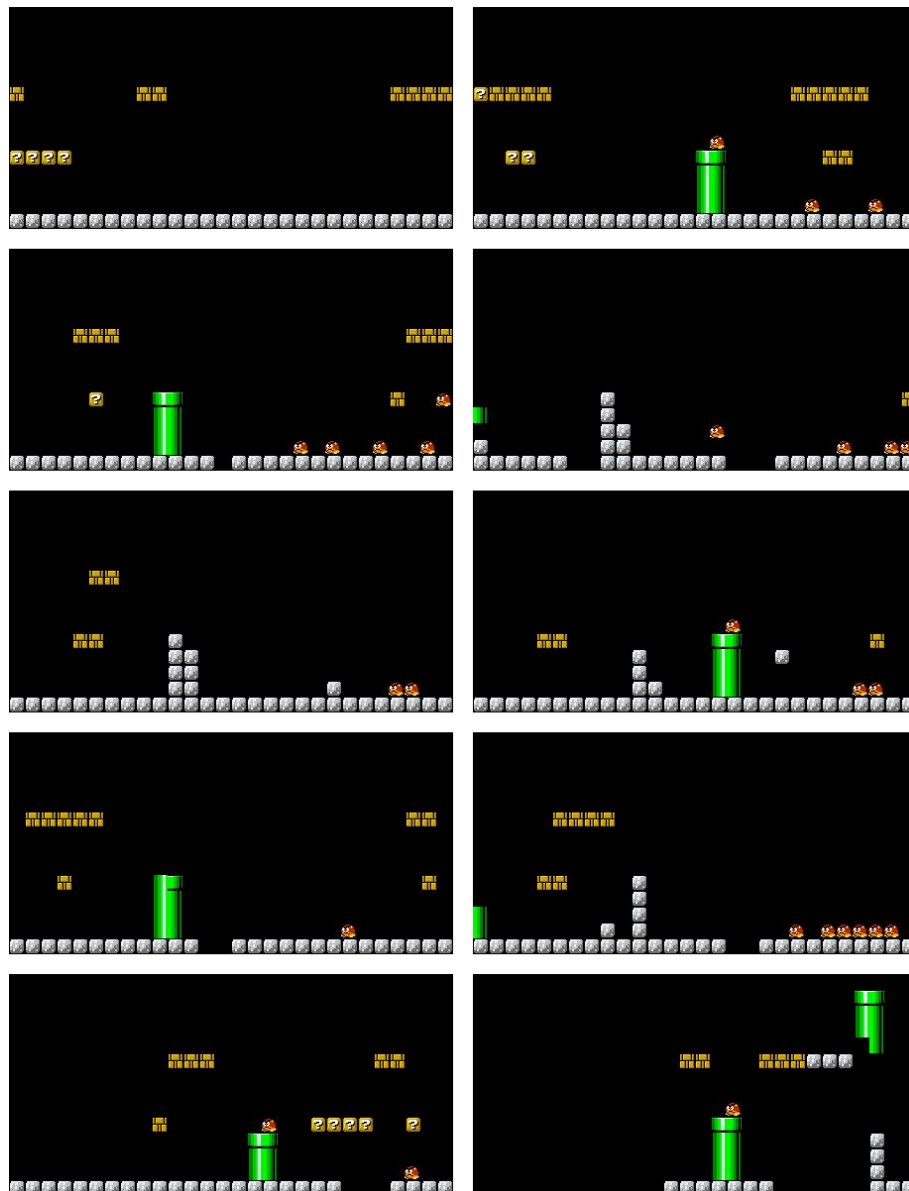


Figura 5.49: 2º Mundo - Níveis Aleatórios

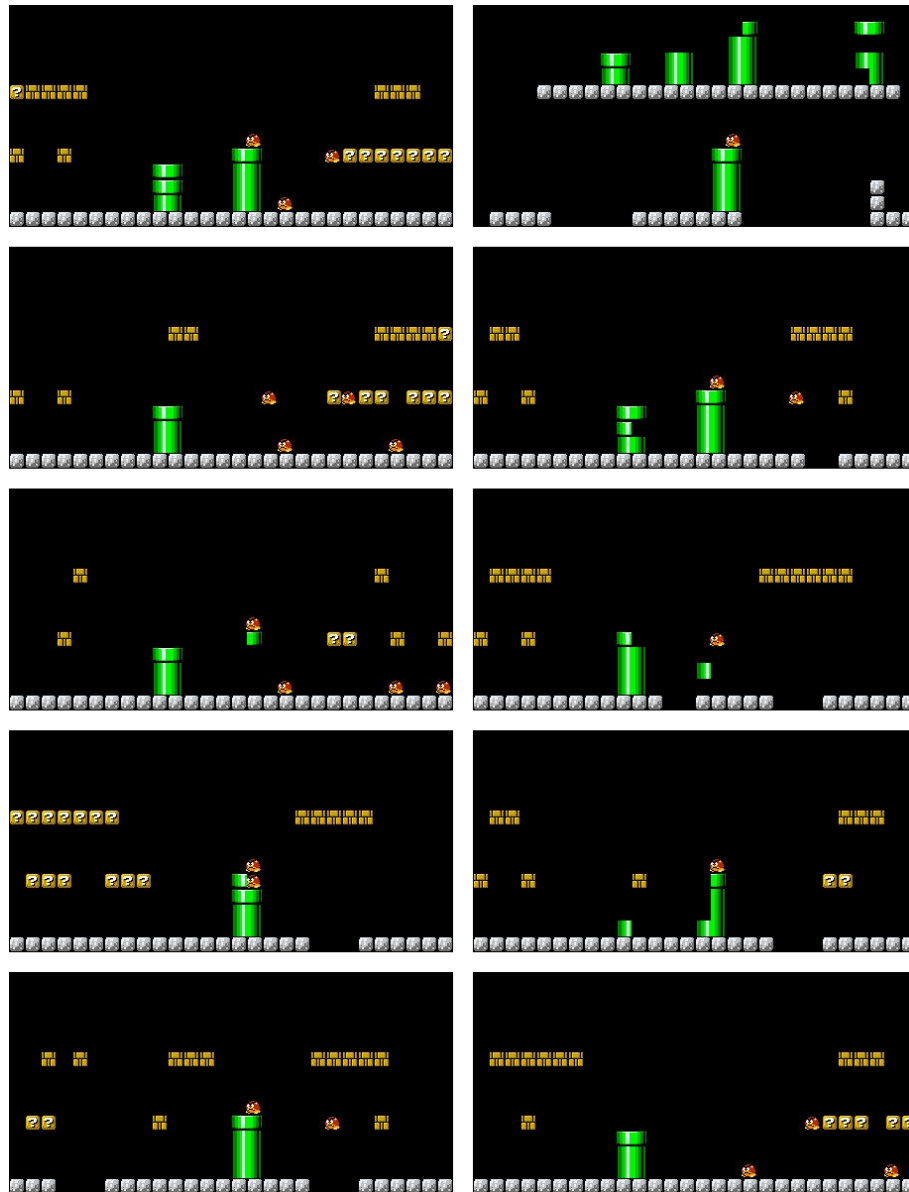


Figura 5.50: 2º Mundo - Níveis Do CMA-ES - Otimizando Canos

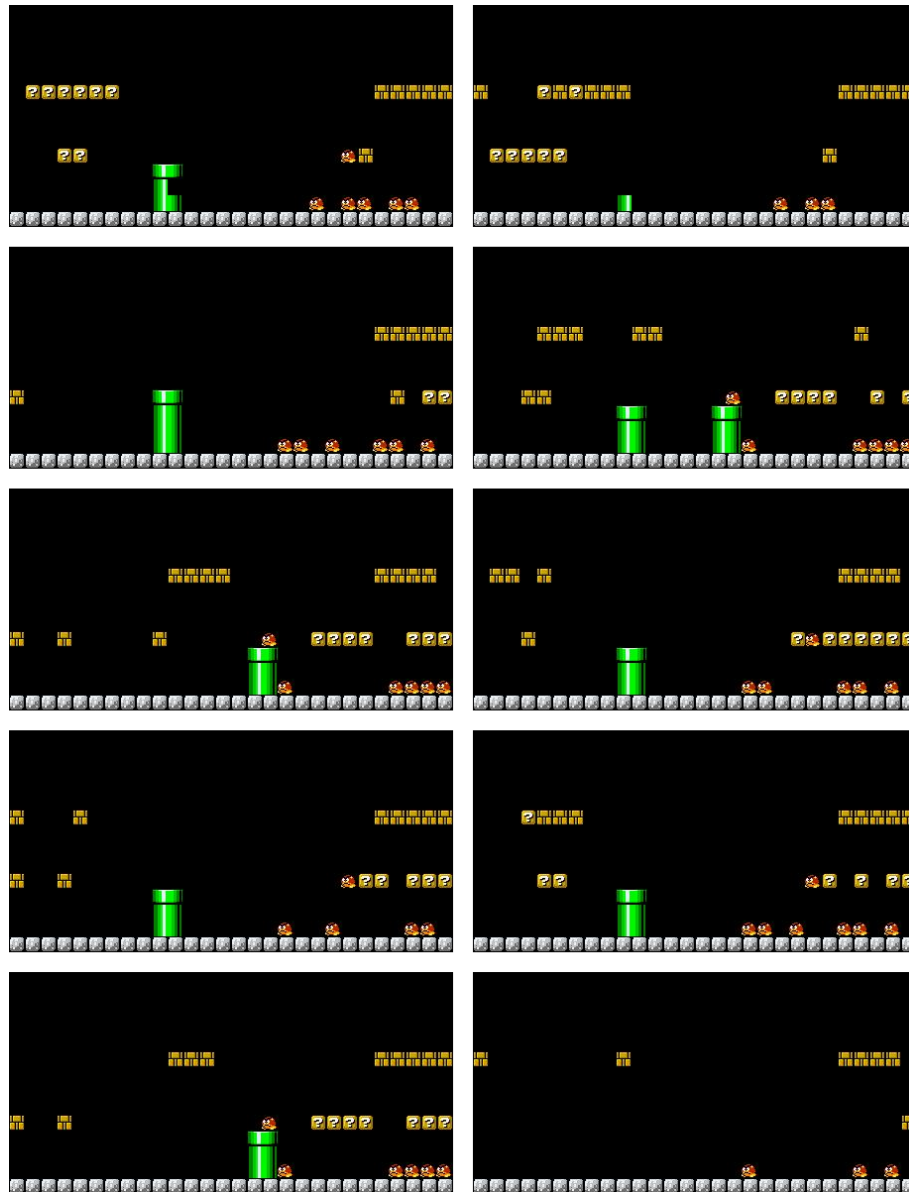


Figura 5.51: 2º Mundo - Níveis Do CMA-ES - Otimizando Inimigos

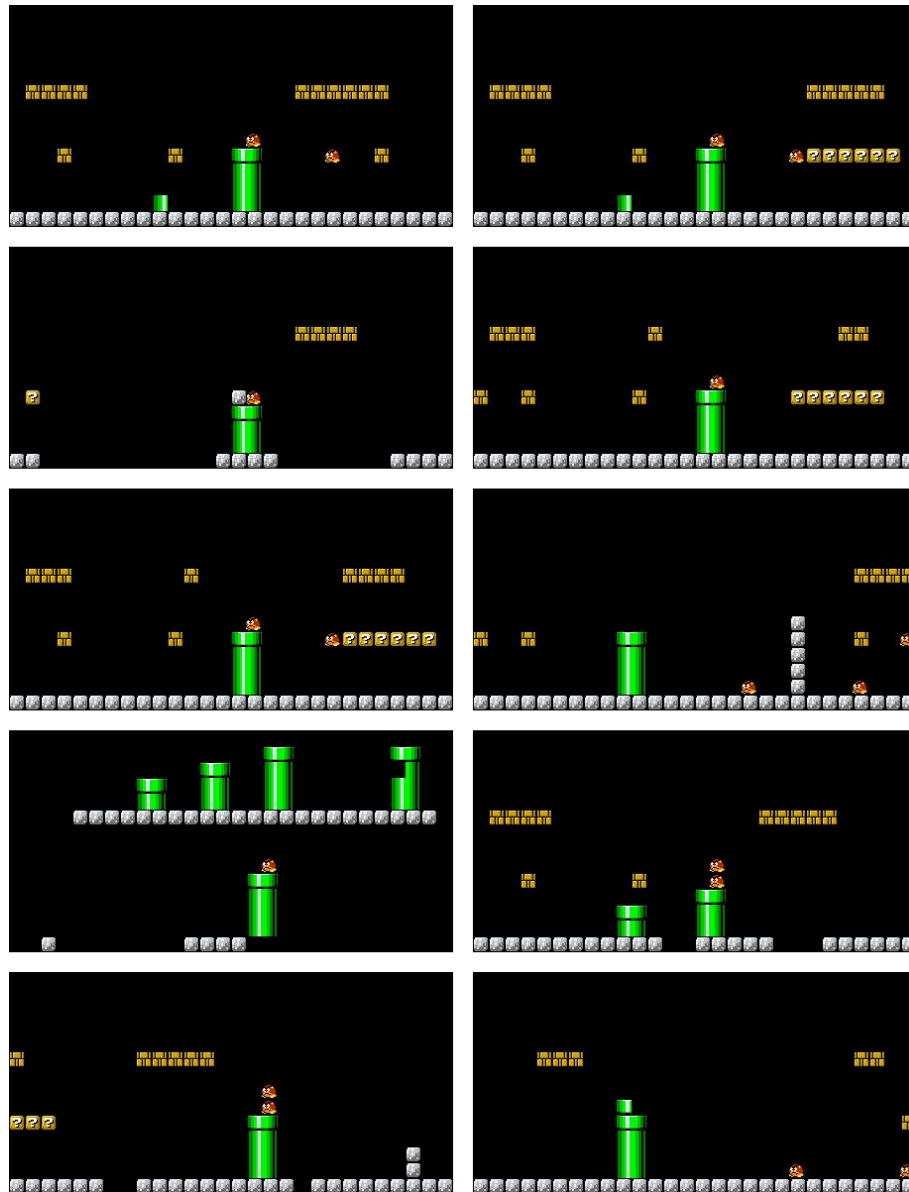


Figura 5.52: 2º Mundo - Níveis Do PSO - Otimizando Canos

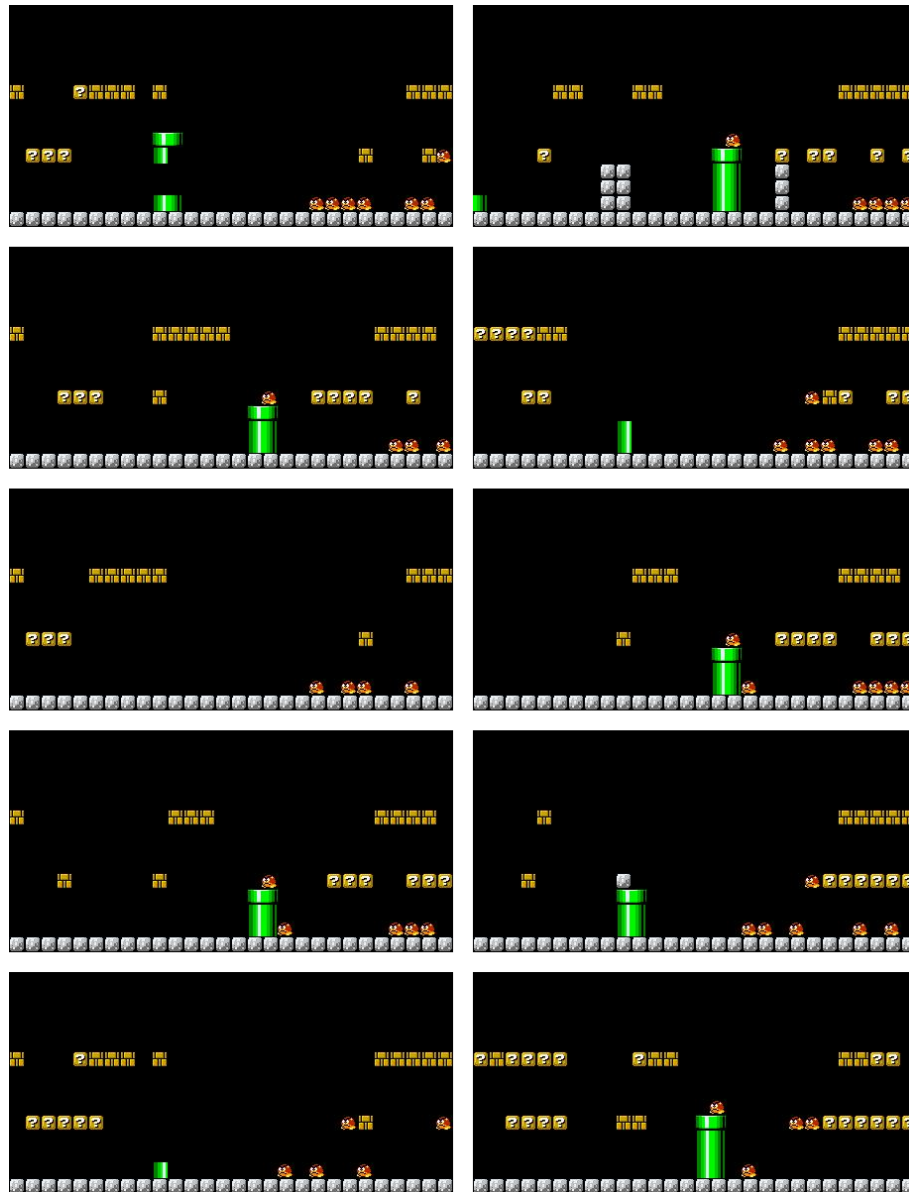


Figura 5.53: 2º Mundo - Níveis Do PSO - Otimizando Inimigos

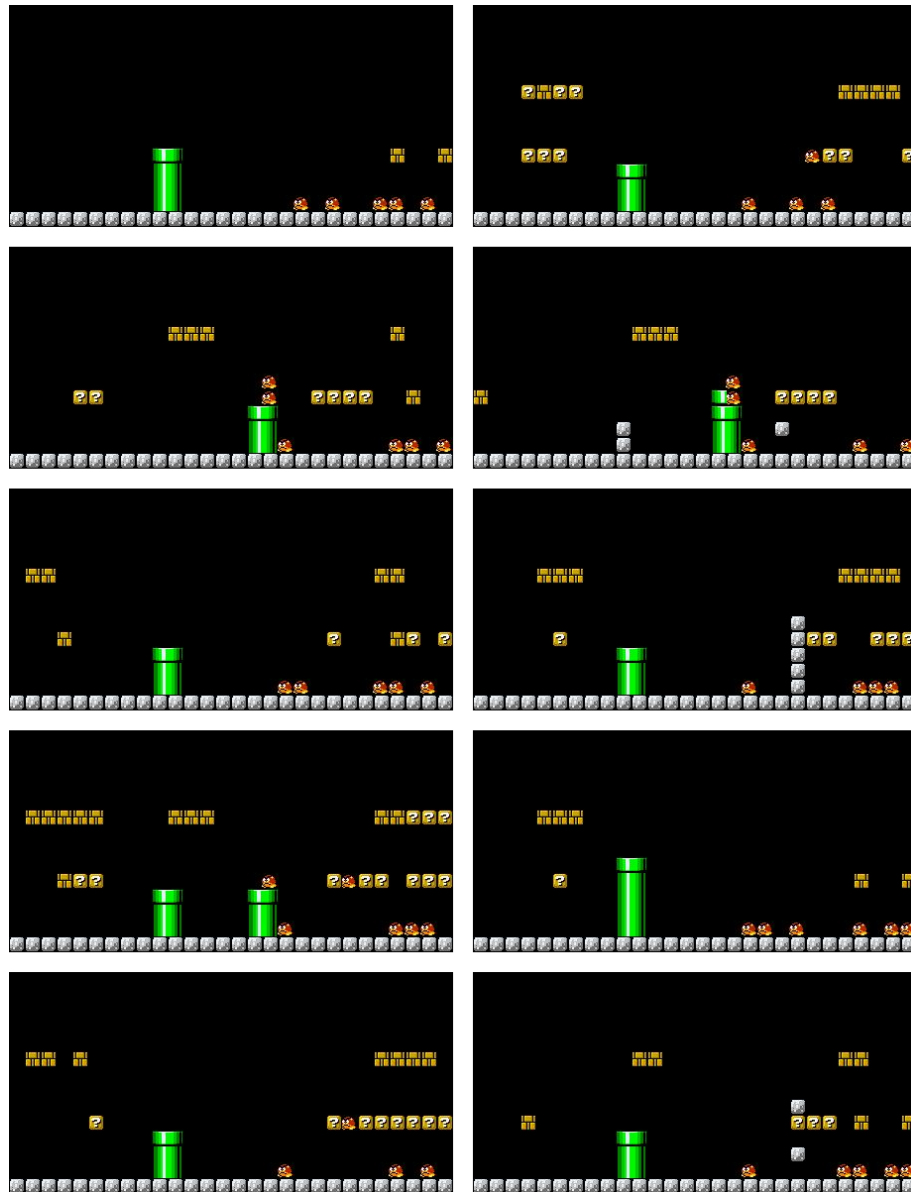


Figura 5.54: 2º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos

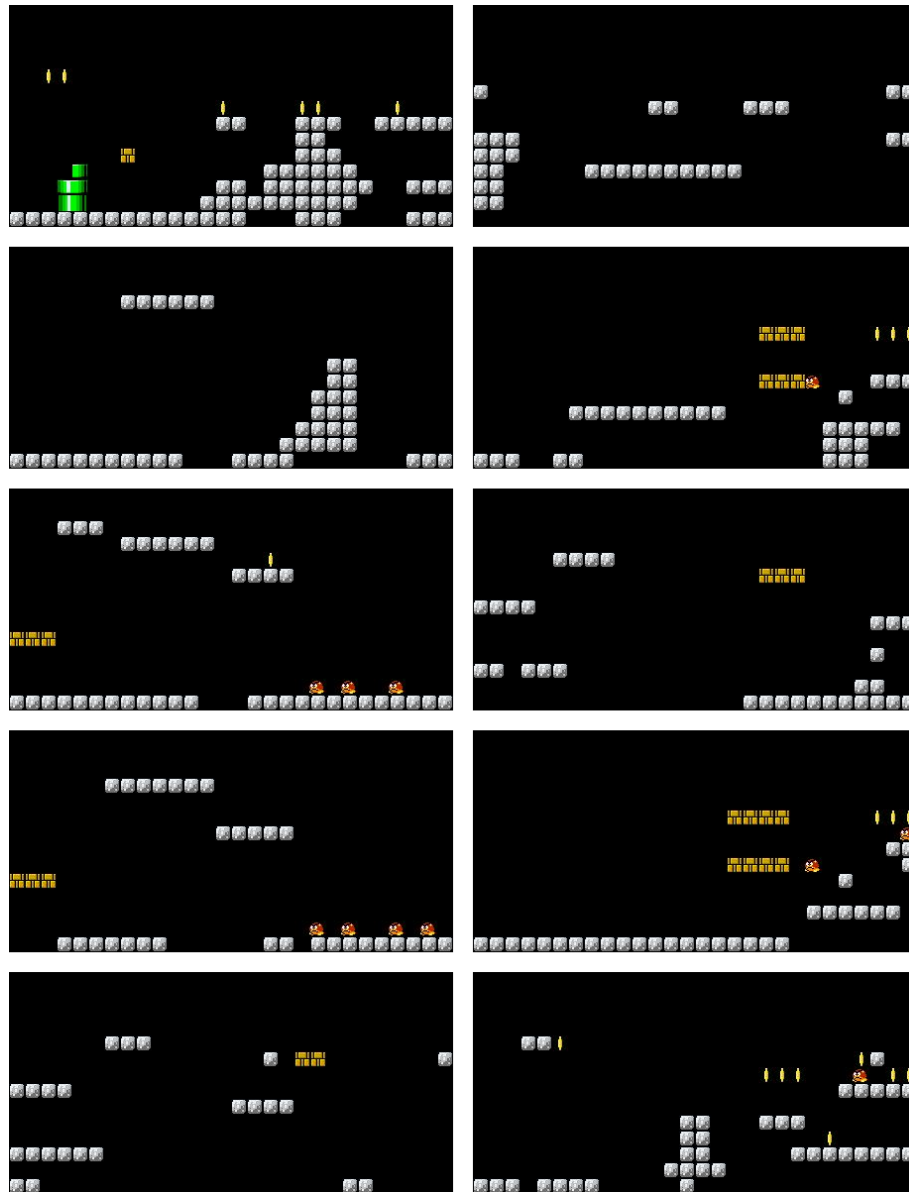


Figura 5.55: 3º Mundo - Níveis Aleatórios

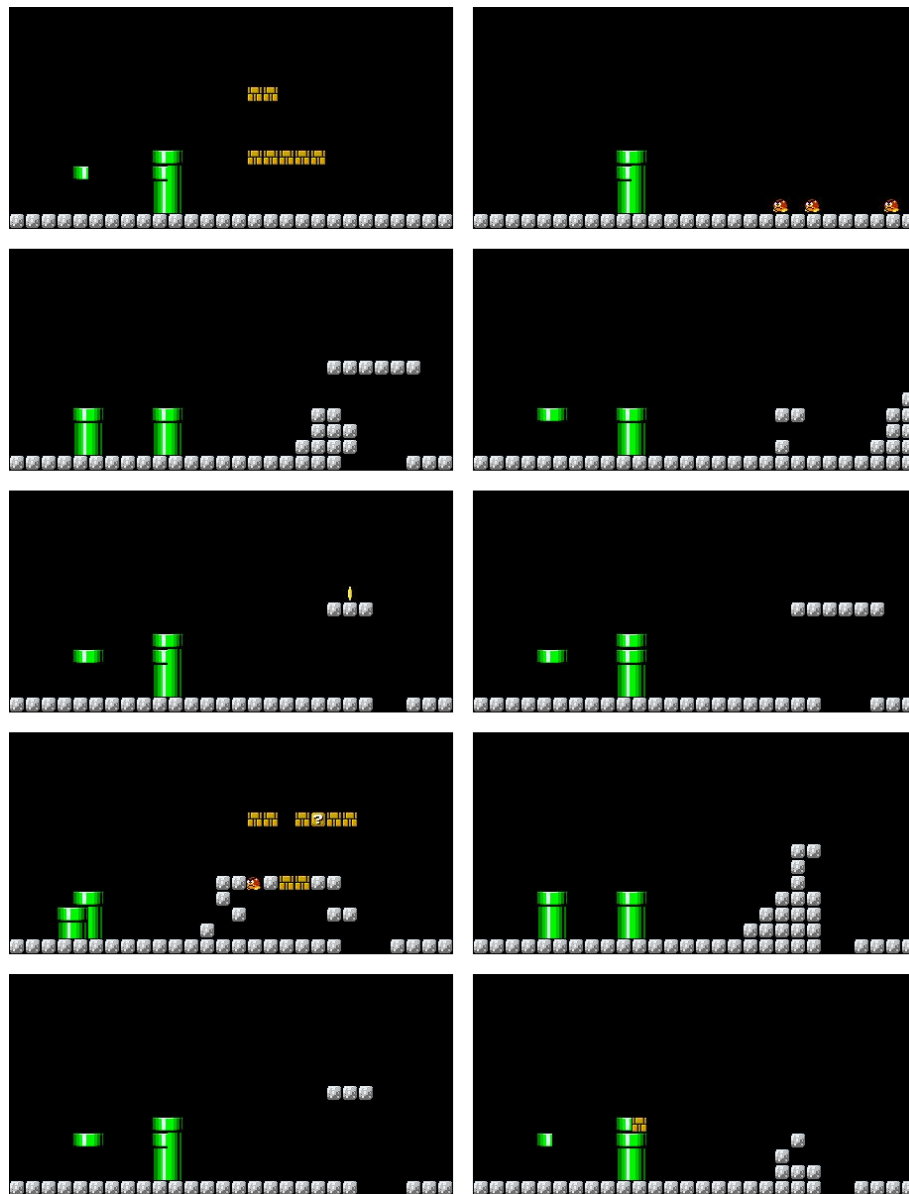


Figura 5.56: 3º Mundo - Níveis Do CMA-ES - Otimizando Canos

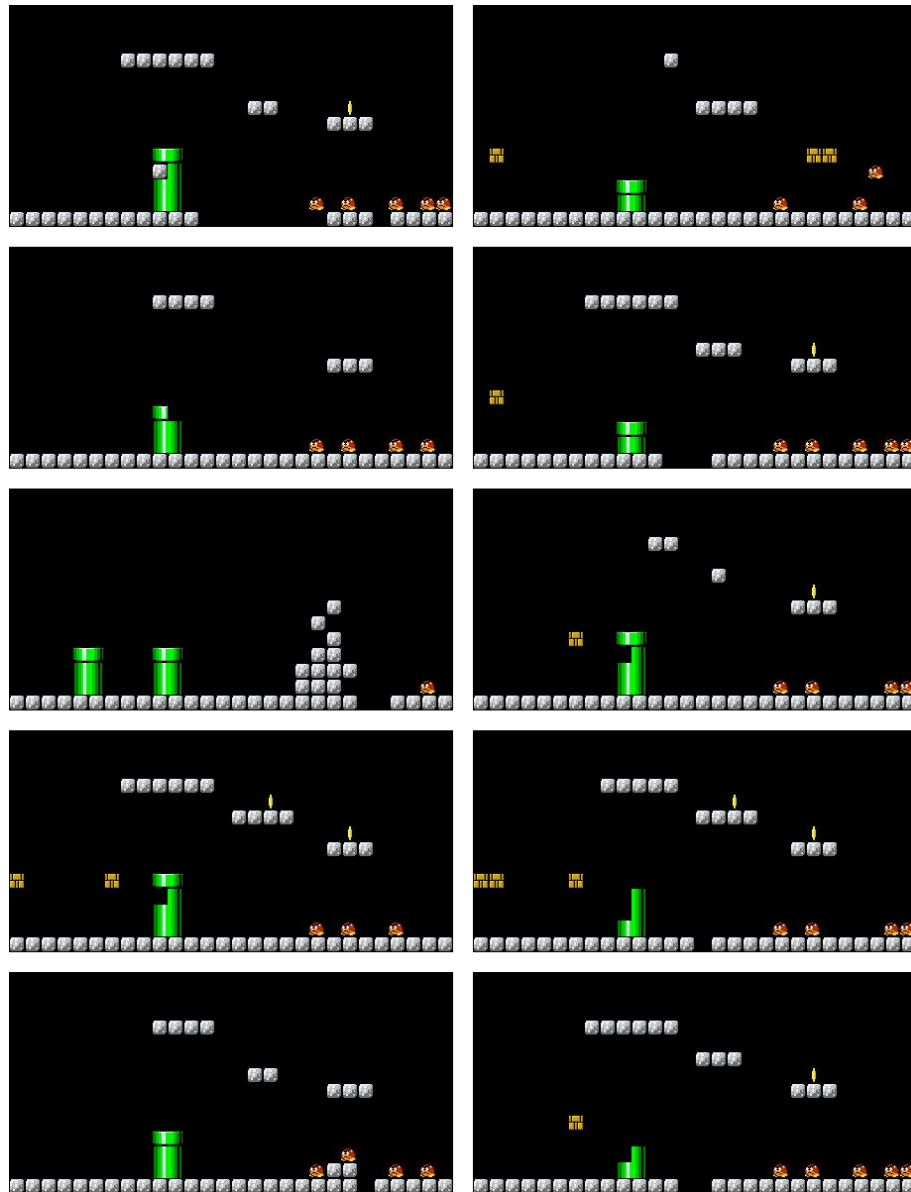


Figura 5.57: 3º Mundo - Níveis Do CMA-ES - Otimizando Inimigos

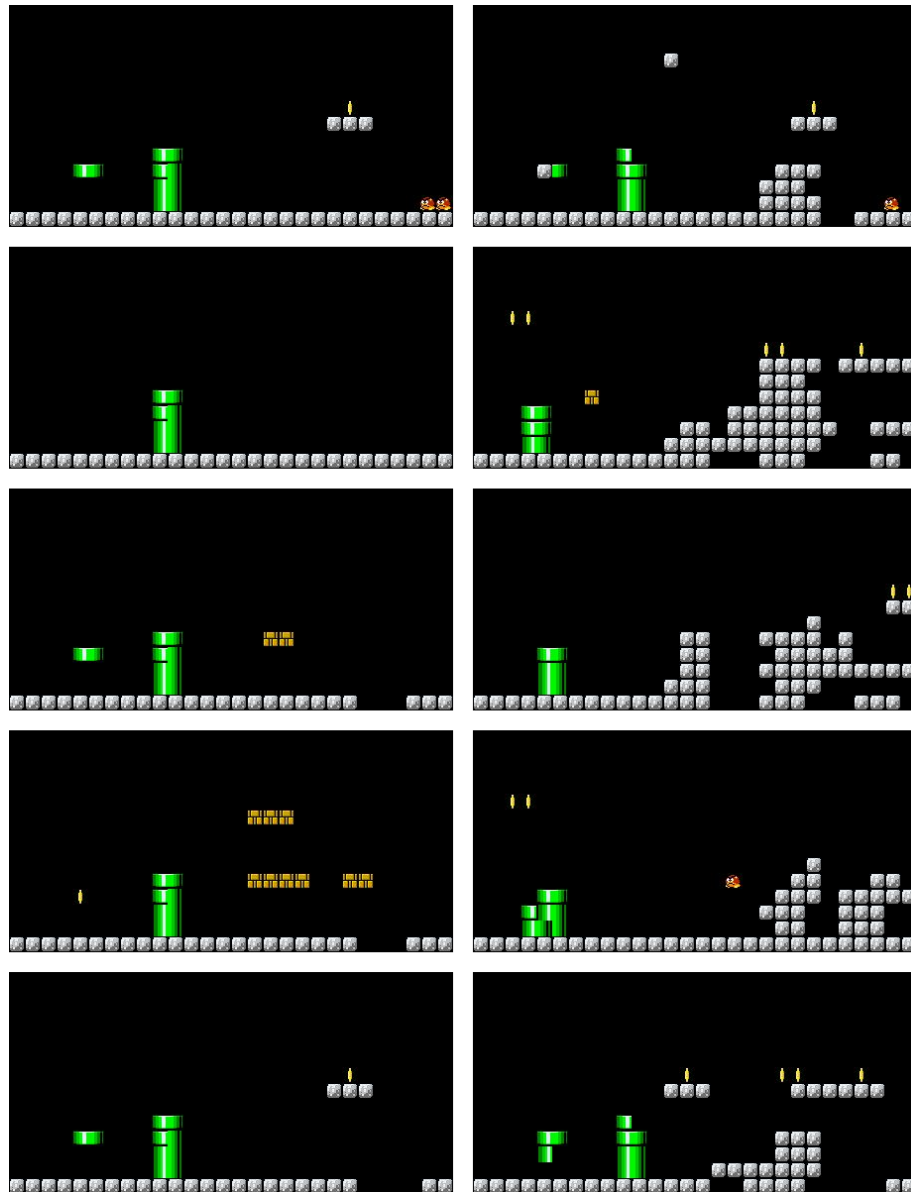


Figura 5.58: 3º Mundo - Níveis Do PSO - Otimizando Canos

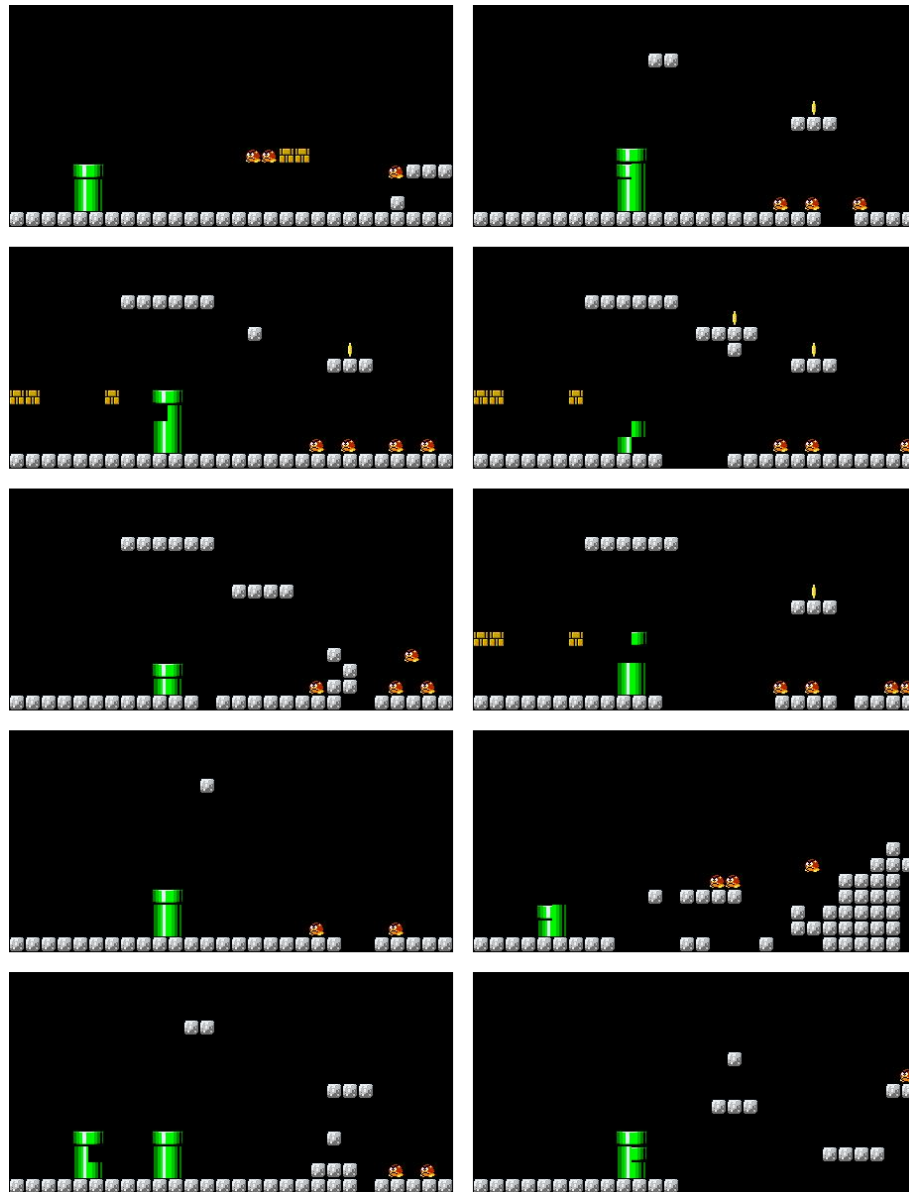


Figura 5.59: 3º Mundo - Níveis Do PSO - Otimizando Inimigos

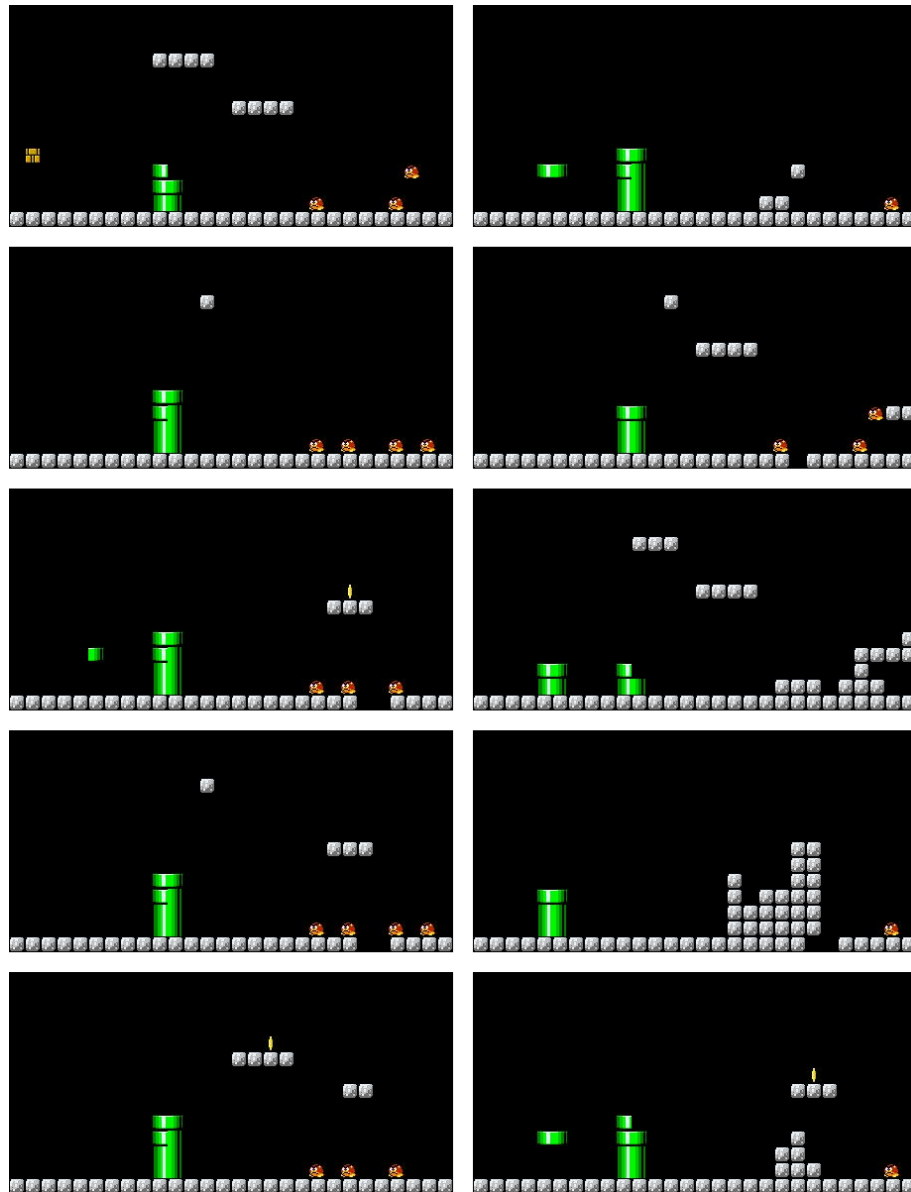


Figura 5.60: 3º Mundo - Níveis Do NSGA-II - Otimizando Canos e Inimigos

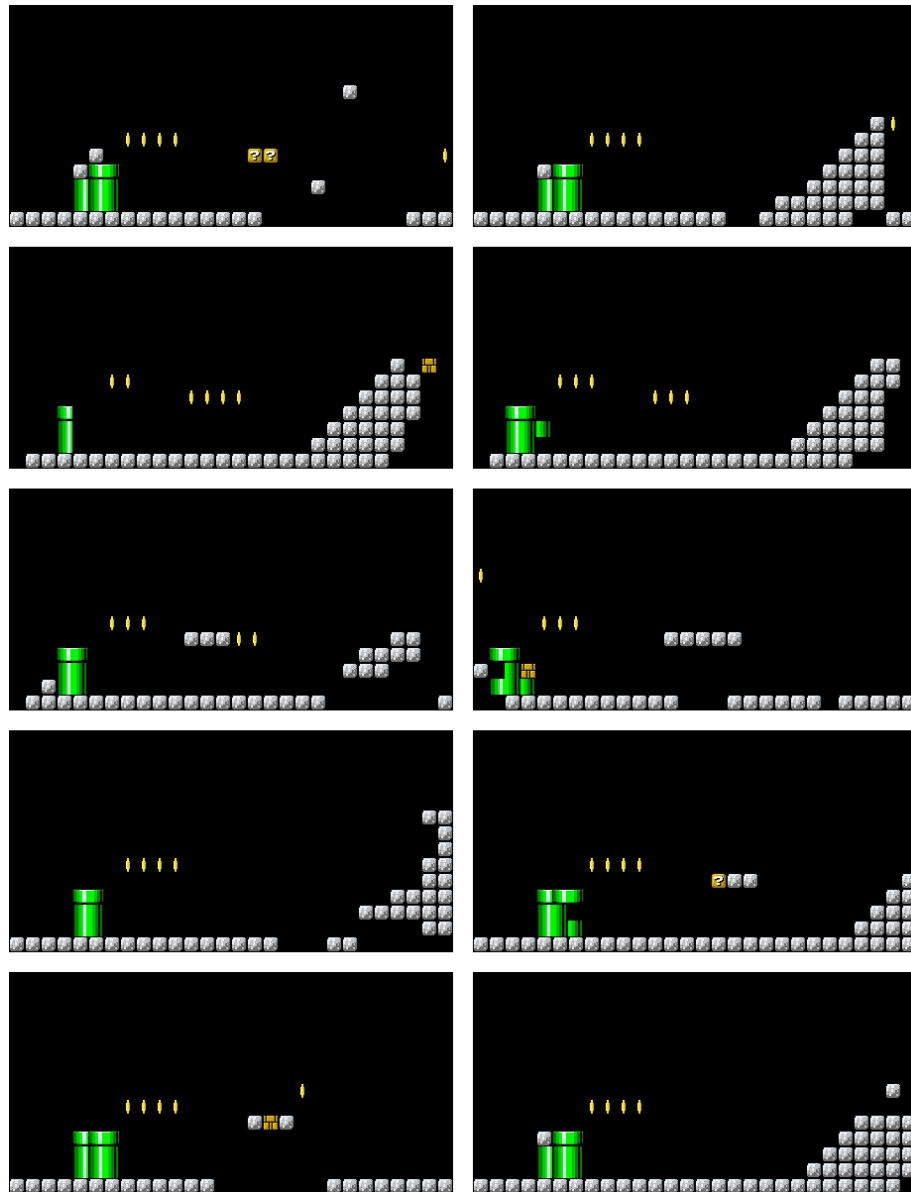


Figura 5.75: 6º Mundo - Níveis Do CMA-ES - Otimizando Inimigos

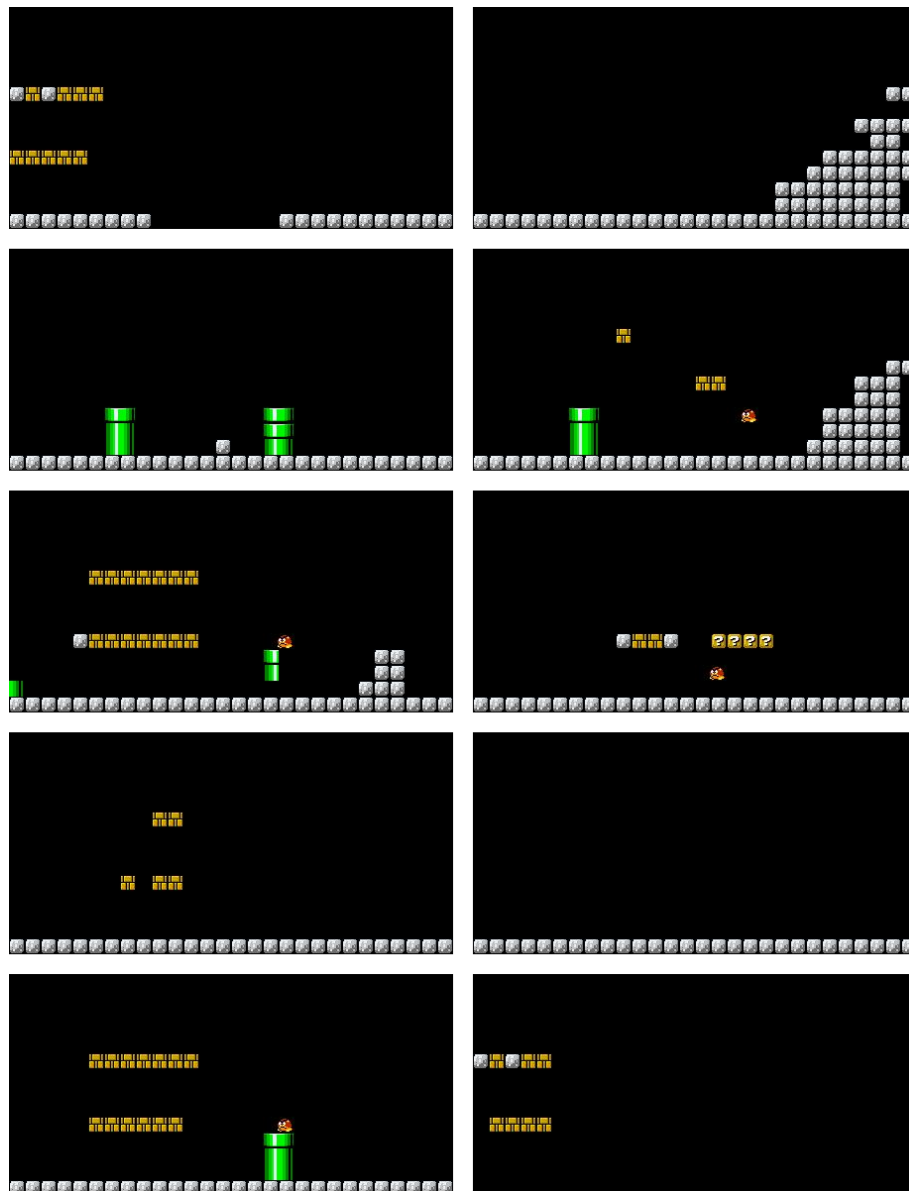


Figura 5.77: 7º Mundo - Níveis Aleatórios

REFERÊNCIAS

- Arjovsky, M., Chintala, S. e Bottou, L. (2017). Wasserstein gan.
- Blank, J. e Deb, K. (2020). pymoo: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509.
- Bontrager, P., Togelius, J. e Memon, N. (2017). Deepmasterprint: Generating fingerprints for presentation attacks.
- Capps, B. e Schrum, J. (2021). Using multiple generative adversarial networks to build better-connected levels for mega man.
- Deb, K. (2002). A fast and elitist multiobjective genetic algorithm: Nsga-ii. Em IEEE, editor, *IEEE Transactions on evolutionary computation*, Vol. 6.
- Fadaeddini, A., Majidi, B. e Eshghi, M. (2018). A case study of generative adversarial networks for procedural synthesis of original textures in video games. Em *2018 2nd National and 1st International Digital Games Research Conference: Trends, Technologies, and Applications (DGRC)*, páginas 118–122.
- Ferreira, L. N., Lelis, L. H. S. e Whitehead, J. (2020). Computer-generated music for tabletop role-playing games.
- Giacomello, E., Lanzi, P. L. e Loiacono, D. (2018). Doom level generation using generative adversarial networks.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. e Bengio, Y. (2014). Generative adversarial networks. *Advances in Neural Information Processing Systems*, 3.
- Hansen, N., Akimoto, Y. e Baudis, P. (2019). CMA-ES/pycma on Github. Zenodo, DOI:10.5281/zenodo.2559634.
- Hansen, N., Müller, S. D. e Koumoutsakos, P. (2003). Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es). *Evolutionary Computation*, 11(1):1–18.
- Hendrikx, M., Meijer, S., Van Der Velden, J. e Iosup, A. (2013). Procedural content generation for games: A survey. *ACM Trans. Multimedia Comput. Commun. Appl.*, 9(1).
- Hong, S., Kim, S. e and, S. K. (2019). Game sprite generator using a multi discriminator gan. *KSI Transactions on Internet and Information Systems*, 13(8):4255–4269.

- Jin, Y., Zhang, J., Li, M., Tian, Y., Zhu, H. e Fang, Z. (2017). Towards the automatic anime characters creation with generative adversarial networks.
- Liu, J., Snodgrass, S., Khalifa, A., Risi, S., Yannakakis, G. N. e Togelius, J. (2020). Deep learning for procedural content generation. *Neural Computing and Applications*, 33(1):19–37.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J. e Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. Em *Advances in Neural Information Processing Systems 32*, páginas 8024–8035. Curran Associates, Inc.
- Rebouças Serpa, Y. e Formico Rodrigues, M. A. (2019). Towards machine-learning assisted asset generation for games: A study on pixel art sprite sheets. Em *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, páginas 182–191.
- Reed, S., Akata, Z., Yan, X., Logeswaran, L., Schiele, B. e Lee, H. (2016). Generative adversarial text to image synthesis.
- Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A. e Togelius, J. (2018). Procedural content generation via machine learning (pcgml).
- Summerville, A. J., Snodgrass, S., Mateas, M. e n’on Villar, S. O. (2016). The vglc: The video game level corpus. *Proceedings of the 7th Workshop on Procedural Content Generation*.
- Togelius, J., Karakovskiy, S. e Baumgarten, R. (2010). The 2009 mario ai competition. páginas 1 – 8.
- Togelius, J., Yannakakis, G. N., Stanley, K. O. e Browne, C. (2011). Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186.
- Torrado, R. R., Khalifa, A., Green, M. C., Justesen, N., Risi, S. e Togelius, J. (2019). Bootstrapping conditional gans for video game level generation.
- Volz, V., Schrum, J., Liu, J., Lucas, S. M., Smith, A. M. e Risi, S. (2018). Evolving mario levels in the latent space of a deep convolutional generative adversarial network. Em *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2018)*, New York, NY, USA. ACM.
- Zhu, J.-Y., Park, T., Isola, P. e Efros, A. A. (2020). Unpaired image-to-image translation using cycle-consistent adversarial networks.